

Claude Certified Architect – Foundations — Full Study Reference

Consolidated, single-file version of this knowledge base for offline reading and printing. Generated from the individual files below — those files are the source of truth; if this file and a source file ever disagree after an update, the source file wins. To regenerate after edits, re-run the concatenation step described in CLAUDE.md.

Contents: Index & blueprint → Foundations → Domain 1–5 → Out-of-scope → Mental map → Slash commands reference.

Claude Certified Architect – Foundations: Exam Reference

Exam: **Claude Certified Architect – Foundations** (CCAR-F), guide effective July 2026 · Target model: `claude-sonnet-4-6` · Target SDKs: `anthropic` (Python) and `claude-agent-sdk` (Python) · Reference current as of August 2026.

This file is an **index** into the reference content — it lists every numbered section with a link into the file under [reference/](#) that actually holds it. The reference is organized to mirror the **exam's own 5-domain blueprint** (plus a foundations file and an out-of-scope file), so studying by domain and studying by file are the same thing.

Links below point to the **file**, not a page-internal anchor — section numbers are prominent headings, so `Ctrl+F` (or your editor's outline view) for `## N.` gets you there reliably even where GitHub's auto-generated anchors are unpredictable (backticks, em-dashes, and slashes in a heading all affect the anchor differently).

Exam Blueprint at a Glance

Domain	Weight	File
1. Agentic Architecture & Orchestration	27%	reference/01-agentic-architecture-and-orchestration.md
2. Tool Design & MCP Integration	18%	reference/02-tool-design-and-mcp-integration.md
3. Claude Code Configuration & Workflows	20%	reference/03-claude-code-configuration-and-workflows.md
4. Prompt Engineering & Structured Output	20%	reference/04-prompt-engineering-and-structured-output.md

Domain	Weight	File
5. Context Management & Reliability	15%	reference/05-context-management-and-reliability.md

Plus: [reference/00-foundations-messages-api.md](#) (prerequisite API mechanics every domain assumes) and [reference/06-out-of-scope.md](#) (explicitly excluded topics — don't study these for this exam).

The 6 Exam Scenarios

The exam draws **4 of these 6** scenarios per sitting. This reference builds running code examples against **all 6**, since which 4 you draw is random:

- 1. **Customer Support Resolution Agent** — `get_customer / lookup_order / process_refund / escalate_to_human`, 80%+ first-contact resolution target
- 2. **Code Generation with Claude Code** — custom slash commands, CLAUDE.md, plan mode vs. direct execution
- 3. **Multi-Agent Research System** — coordinator + web-search/document-analysis/synthesis/report subagents
- 4. **Developer Productivity with Claude** — built-in tools (Read/Write/Bash/Grep/Glob) + MCP servers on unfamiliar codebases
- 5. **Claude Code for Continuous Integration** — automated PR review, test generation, false-positive minimization
- 6. **Structured Data Extraction** — JSON schema validation, edge-case handling, downstream integration

Focus Areas (from the last practice-exam score report)

These task statements scored 0%–50% and got extra depth/worked examples in this pass — if short on time, review these sections first (§ = section number; find it with `Ctrl+F "## N."` in the linked file):

Task Statement	Section (s)	File
3.1 — Enforcement (hooks/permissions) vs. CLAUDE.md placement	§46, §47	03-claude-code-configuration-and-workflows.md
3.1 — Choosing CLAUDE.md vs. rules vs. skills vs. hooks vs. permissions	§46	03-claude-code-configuration-and-workflows.md
1.5 — <code>PostToolUse</code> hooks enforcing code quality independent of instruction-following	§48	03-claude-code-configuration-and-workflows.md
5.2 — Escalation: honor explicit request vs. autonomous resolution	§58	05-context-management-and-reliability.md
4.4 — Feedback-loop metadata (<code>detected_pattern</code>) improving prompts/schemas over time	§56	04-prompt-engineering-and-structured-output.md
2.2 — Structured, type-specific MCP tool error responses	§42	02-tool-design-and-mcp-integration.md
1.6 — Adaptive task decomposition vs. fixed sequences	§38	01-agentic-architecture-and-orchestration.md
1.1 — <code>stop_reason</code> loop termination anti-patterns	§40	01-agentic-architecture-and-orchestration.md

Task Statement	Section (s)	File
3.5 — Iterative refinement (examples, targeted feedback, batching)	§50	03-claude-code-configuration-and-workflows.md
2.5 — Systematic Grep/Glob/Read exploration under context constraints	§45	02-tool-design-and-mcp-integration.md
2.4 — MCP server scope: project (team) vs. user (personal/experimental)	§44	02-tool-design-and-mcp-integration.md
5.2 — Self-reported confidence / sentiment are unreliable escalation proxies	§25	05-context-management-and-reliability.md
5.5 — Confidence/characteristic-based review routing vs. random sampling	§61	05-context-management-and-reliability.md
2.1 — Tool description disambiguation for semantically similar tools	§41	02-tool-design-and-mcp-integration.md

Contents

Next available section number: 64 — when adding a new section, use this number and bump it here. (Numbers 52, 53, and 57 are intentionally retired/unused — see notes below; gaps in the sequence are expected and fine.)

Foundations — Messages API Core Mechanics — [reference/00-foundations-messages-api.md](#)

Prerequisite building blocks assumed by every domain: tool choice, content blocks, stop reasons, built-in/local tools, models & pricing, message roles, request/response shape, the multi-turn tool loop, `maxTurns`, and API statelessness.

- §1 — Tool Choice
- §2 — Content Block Types
- §3 — Stop Reasons
- §4 — Built-in (Standard) Tools
- §5 — Local Tool Definition
- §6 — Current Models and Pricing
- §7 — Session Resumption — The API Is Stateless
- §8 — Message Roles
- §9 — Top-level Request Parameters
- §10 — Response Object Fields
- §11 — Multi-turn Tool Loop Pattern
- §13 — `maxTurns`

Domain 1: Agentic Architecture & Orchestration (27%) — [reference/01-agentic-architecture-and-orchestration.md](#)

Agentic loop anti-patterns, hub-and-spoke orchestration, the `Task / Agent` tool & `AgentDefinition`, multi-step workflow enforcement/handoff, Agent SDK hooks, task decomposition strategy, session management,

and `fork_session`.

- §12 — Session Management (Claude Code CLI and Agent SDK)
- §21 — Agent SDK / Claude Code Hooks — Tool Call Interception and Data Normalization
- §24 — Multi-Agent Architecture — Hub-and-Spoke Pattern
- §36 — Subagent Spawning — the `Task` / Agent Tool and `AgentDefinition`
- §37 — `fork_session` — Branching a Session (Agent SDK)
- §38 — Task Decomposition Strategies for Complex Workflows
- §39 — Multi-Step Workflows — Enforcement and Handoff Patterns
- §40 — Agentic Loop Anti-Patterns — What NOT to Check for Termination

Domain 2: Tool Design & MCP Integration (18%) — [reference/02-tool-design-and-mcp-integration.md](#)

Tool description quality & disambiguation, structured MCP error responses, distributing tools across agents, MCP server scope/config/resources, and systematic use of built-in tools.

- §23 — Built-in Tool Selection — Bash vs. Read/Glob/Grep, and Edit vs. Write
- §26 — MCP Server Resources: @ Mention Reference Syntax (*covered within §44*)
- §27 — MCP Config: Environment Variable Expansion in `.mcp.json` (*covered within §44*)
- §28 — MCP Server Authentication (*covered within §44*)
- §29 — MCP Server Config: Scope Precedence for Duplicate Server Names (*covered within §44*)
- §41 — Tool Description Quality and Disambiguation
- §42 — Structured Error Responses for MCP Tools
- §43 — Distributing Tools Appropriately Across Agents
- §44 — Configuring MCP Servers at the Correct Scope
- §45 — Systematic Codebase Exploration — Grep, Glob, Read Together

Domain 3: Claude Code Configuration & Workflows (20%) — [reference/03-claude-code-configuration-and-workflows.md](#)

Choosing the right configuration mechanism, enforcement vs. guidance, `PostToolUse` quality gates, slash commands & skills, path-scoped rules, permission modes, CLI supporting mechanics, plan mode, iterative refinement, and CI/CD integration.

- §16 — Custom Slash Commands and Skills — Scope and Configuration
- §18 — Path-Scoped Rules and Symlinks in Claude Code
- §20 — Claude Code Permission Modes
- §22 — `.claude/rules/` and Path-Scoped Loading (incl. `/memory`)
- §32 — Blocking Bash in CI — `--disallowedTools`
- §33 — Piped Stdin Size Limit in Claude Code — 10 MB Cap
- §34 — `claudeMdExcludes` — Personal Monorepo Filtering
- §35 — CLAUDE.md @path Import Syntax
- §46 — Choosing the Right Claude Code Configuration Mechanism
- §47 — Enforcement Layer vs. Guidance Layer — the Deterministic/Probabilistic Split
- §48 — `PostToolUse` Hooks for Automatic Code-Quality Enforcement
- §49 — Plan Mode vs. Direct Execution
- §50 — Iterative Refinement Techniques
- §51 — Integrating Claude Code into CI/CD Pipelines

(§52 is unused — reserved slot from an earlier draft, never assigned.)

Domain 4: Prompt Engineering & Structured Output (20%) — [reference/04-prompt-engineering-and-structured-output.md](#)

Explicit criteria & false-positive reduction, XML tag structuring, few-shot prompting, `tool_use` + JSON schema enforcement, validation/retry/feedback loops, the Message Batches API, and multi-instance/multi-pass review.

- §14 — Multi-Instance and Multi-Pass Review Architectures (*includes the former "multi-pass" content once drafted separately as §57 — folded in; §57 is retired*)
- §15 — Structured Output — Handling Missing Data in Tool Schemas (*covered within §55*)
- §19 — Prompt Structuring — XML Tags for Category Isolation
- §30 — Message Batches API and Batch Processing Strategy
- §31 — Explicit Criteria to Improve Precision and Reduce False Positives (*§53 folded in — retired, same task statement*)
- §54 — Few-Shot Prompting for Output Consistency
- §55 — Enforcing Structured Output with `tool_use` and JSON Schemas
- §56 — Validation, Retry, and Feedback Loops for Extraction Quality

Domain 5: Context Management & Reliability (15%) — [reference/05-context-management-and-reliability.md](#)

Long-session context preservation, large-codebase exploration, escalation/ambiguity resolution, error propagation across multi-agent systems, human review/confidence calibration, and information provenance in multi-source synthesis.

- §17 — Managing Conversation Context to Preserve Critical Information
- §25 — Agent Escalation Design — Self-Reported Confidence, in Detail
- §58 — Escalation Triggers and Ambiguity Resolution
- §59 — Error Propagation Strategies Across Multi-Agent Systems
- §60 — Context Management in Large Codebase Exploration
- §61 — Human Review Workflows and Confidence Calibration
- §62 — Information Provenance and Uncertainty in Multi-Source Synthesis

Out-of-Scope Exam Topics — [reference/06-out-of-scope.md](#)

Fine-tuning, billing/auth protocol details, cloud-provider specifics, computer use, vision, streaming, rate limits, benchmarking, prompt-caching/tokenization internals, and other topics the exam guide explicitly excludes.

- §63 — Out-of-Scope Topics Quick Reference

Foundations — Messages API Core Mechanics

Prerequisite building blocks used across every exam domain: tool choice, content blocks, stop reasons, built-in/local tools, current models, message roles, request/response shape, the agentic tool loop, `maxTurns`, and why the API is stateless. Not itself one of the 5 blueprint domains — every domain file below assumes this one.

Scope note: examples target `claude-sonnet-4-6` (the exam's documented model) and the Python `anthropic` SDK, per [CLAUDE.md](#).

1. Tool Choice

Controls whether and how Claude uses tools.

```
tool_choice={"type": "auto"}           # Claude decides (default)
tool_choice={"type": "any"}            # Claude must call some tool
tool_choice={"type": "tool", "name": "calculator"} # Force a specific tool
tool_choice={"type": "none"}           # Disable all tools
```

`auto` vs `any` — the critical distinction

Mode	Must call a tool?	Can respond with plain text?	Claude picks which tool?
<code>auto</code>	❌ No — optional	✅ Yes	✅ Yes
<code>any</code>	✅ Yes — guaranteed	❌ No	✅ Yes
<code>tool</code>	✅ Yes — guaranteed	❌ No	❌ No (forced)
<code>none</code>	❌ Prohibited	✅ Yes	—

`auto` — Claude decides whether to call a tool *or skip all tools entirely* and reply with plain text.

`any` — Claude must call one of the registered tools. It cannot reply with plain text. It picks which tool fits the input best using its semantic understanding.

When to use `tool` (force a specific tool)

Use `{"type": "tool", "name": "..."}` when a **specific tool must run on this turn** — regardless of what other tools are registered. Exam-relevant case: a structured-extraction pipeline (Domain 4 / Scenario 6) where `extract_metadata` must run before any enrichment tool.

```
# Pipeline step 1: extract_metadata must run before enrichment
response = client.messages.create(
    model="claude-sonnet-4-6",
    tools=[extract_metadata_tool, translate_text_tool, summarize_document_tool],
    tool_choice={"type": "tool", "name": "extract_metadata"}, # only this fires
    messages=[{"role": "user", "content": document_text}]
)
```

`any` is wrong here: it guarantees *some* tool fires, but Claude could pick a different one first, violating the dependency order.

Enforcing tool ordering across a pipeline

`tool_choice` is a **per-request** setting, not a session-wide execution plan. To guarantee ordering when all tools are eventually needed, split into multiple turns:

```
# Turn 1 – force the prerequisite tool
resp1 = client.messages.create(
    tools=[extract_metadata_tool, translate_text_tool, summarize_document_tool],
    tool_choice={"type": "tool", "name": "extract_metadata"},
    messages=[{"role": "user", "content": doc}]
)
metadata_result = run_tool(resp1) # execute extract_metadata locally

# Turn 2 – pass metadata back; Claude calls remaining tools freely
resp2 = client.messages.create(
    tools=[translate_text_tool, summarize_document_tool],
    tool_choice={"type": "any"}, # must call one of the remaining tools
    messages=[
        {"role": "user", "content": doc},
        {"role": "assistant", "content": resp1.content},
        {"role": "user", "content": [{"type": "tool_result", **metadata_result}]},
    ]
)
```

When to use `any` instead of `auto`

Use `any` when you need **guaranteed structured output for every input** (Domain 4 / Scenario 6) and cannot tolerate a plain-text fallback.

```
# Support ticket router – must always call one extraction tool, never reply with text
response = client.messages.create(
    model="claude-sonnet-4-6",
    tools=[bug_report_tool, feature_request_tool, billing_issue_tool],
    tool_choice={"type": "any"}, # ← guarantees one tool fires; Claude picks which
    messages=[{"role": "user", "content": ticket_text}]
)
```

Why `auto` fails here: an ambiguous ticket might cause Claude to reply *"I'm not sure how to classify this"* in plain text — breaking the guarantee of a structured result for every ticket.

Extended thinking — `tool_choice` compatibility

When extended thinking is enabled, only `"auto"` and `"none"` are compatible `tool_choice` values:

tool_choice value	Compatible with extended thinking?
<code>{"type": "auto"}</code>	✔ Yes
<code>{"type": "none"}</code>	✔ Yes
<code>{"type": "any"}</code>	✘ No — forces tool call, incompatible
<code>{"type": "tool", "name": "..."} </code>	✘ No — forces tool call, incompatible

2. Content Block Types

Appear in `response.content` or in message `content` arrays.

```
{
  "type": "text",
  "text": "Hello!"
} # Claude's text

{
  "type": "tool_use",
  "id": "toolu_01abc...",
  "name": "calculator",
  "input": {
    "expression": "2+2"
  }
} # Claude calling a tool

{
  "type": "tool_result",
  "tool_use_id": "toolu_01abc...",
  "content": "4"
} # your result back to Claude

{
  "type": "image",
  "source": {
    "type": "base64",
    "media_type": "image/jpeg",
    "data": "<b64>"
  }
}

{
  "type": "document",
  "source": {
    "type": "base64",
    "media_type": "application/pdf",
    "data": "<b64>"
  }
}
```

3. Stop Reasons

`response.stop_reason` — why Claude stopped generating:

Value	Meaning	What to do next
"end_turn"	Claude finished naturally (including asking questions)	Get user input → append → call API again
"tool_use"	Claude wants to call a tool	Run tool → return <code>tool_result</code> → call API again
"max_tokens"	Hit the <code>max_tokens</code> ceiling mid-response	Optionally continue by appending partial response
"stop_sequence"	Hit a custom stop sequence you defined	Application-specific
"pause_turn"	A long-running turn (e.g. an extended server-tool-use turn) was paused so it can be continued	Send the next request to resume it
"refusal"	Safety classifier declined. HTTP 200	Check <code>stop_details.category</code> ; handle gracefully
"model_context_window_exceeded"	The response filled the model's context window mid-generation	Treat as truncated, same as <code>max_tokens</code>

There is no "waiting for user input" stop reason. The API is pure request-response. When Claude asks a clarifying question mid-task, `stop_reason` is still `"end_turn"` — indistinguishable from a final answer without parsing text content.

Better pattern — a `request_clarification` tool for structural detectability:

```
{
  "name": "request_clarification",
  "description": "Ask the user a clarifying question before proceeding",
  "input_schema": {
    "type": "object",
  }
}
```

```

    "properties": {"question": {"type": "string"}, "reason": {"type": "string"}},
    "required": ["question"]
  }
}

```

```

while response.stop_reason == "tool_use":
    tool_call = next(b for b in response.content if b.type == "tool_use")
    if tool_call.name == "request_clarification":
        result = input(f"Claude asks: {tool_call.input['question']}\n> ")
    else:
        result = run_my_tool(tool_call.name, tool_call.input)
    # append tool_result and continue loop ...

```

Approach	stop_reason	Detectable without parsing?
Claude asks in plain text	end_turn	✗
request_clarification tool	tool_use	✓

4. Built-in (Standard) Tools

Anthropic-managed tools — no `input_schema` needed, no result handling required.

```

{"type": "web_search_20260209", "name": "web_search"}           # Web search
{"type": "web_fetch_20260209", "name": "web_fetch"}           # Web fetch
{"type": "bash_20250124", "name": "bash"}                     # Run bash commands
{"type": "text_editor_20250728", "name": "str_replace_based_edit_tool"} # Edit files
{"type": "computer_20251124", "name": "computer"}             # Control computer (out of
scope — see §63)
{"type": "code_execution_20260521", "name": "code_execution"} # Server-side code
execution

```

	Built-in	Local
Has <code>type</code> field	✓ Yes (versioned)	✗ No
Has <code>input_schema</code>	✗ No	✓ Yes
Who runs the tool	Anthropic	You
Needs second turn	✗ No	✓ Yes

Type strings are versioned and change between API versions — note the version suffix whenever it appears in a question (e.g. `bash_20250124`).

5. Local Tool Definition

You define the schema; you run the tool and return the result. This is the shape used for every custom/MCP tool in the exam scenarios (`get_customer`, `lookup_order`, `process_refund`, etc.):

```
{
  "name": "get_customer",
  "description": "Look up a verified customer record by email or phone. Returns a customer_id used by all other support tools. Call this FIRST for any request that will touch order or billing data.",
  "input_schema": {
    "type": "object",
    "properties": {
      "identifier": {"type": "string", "description": "Customer email or phone number"}
    },
    "required": ["identifier"]
  }
}
```

6. Current Models and Pricing

Model	Model ID	Context	Input \$/1M	Output \$/1M
Claude Opus 5	claude-opus-5	1M	\$5.00	\$25.00
Claude Sonnet 5	claude-sonnet-5	1M	\$2.00	\$10.00
Claude Sonnet 4.6	claude-sonnet-4-6	1M	\$3.00	\$15.00
Claude Haiku 4.5	claude-haiku-4-5	200K	\$1.00	\$5.00

Use **exact model ID strings** — never append date suffixes. The exam guide (effective July 2026) documents `claude-sonnet-4-6` as the target model; this reference keeps that scope for consistency. Newer families (Opus 5, Sonnet 5, Fable 5/5.1) exist in the current live API but are **not** the exam's documented scope — see [06-out-of-scope.md](#).

7. Session Resumption — The API Is Stateless

The Anthropic API has no session ID parameter. The model only sees what is in `messages[]` for the current request.

```
# ❌ This does NOT work — session_id is not an API parameter
client.messages.create(model="claude-sonnet-4-6", session_id="sess_abc123", messages=[...])

# ✅ Correct — retrieve prior history from YOUR storage and inject it
prior_history = db.load_conversation("sess_abc123")
client.messages.create(
  model="claude-sonnet-4-6",
  messages=[*prior_history, {"role": "user", "content": "What's my risk tolerance?"}]
)
```

Correct session-resume pattern

```
def save_turn(session_id, messages):
    db.store(session_id, messages)
```

```
def resume_session(session_id, new_message):
    messages = db.load(session_id) + [{"role": "user", "content": new_message}]
    response = client.messages.create(model="claude-sonnet-4-6", max_tokens=1024, messages=messages)
    messages.append({"role": "assistant", "content": response.content})
    save_turn(session_id, messages)
    return response
```

Claude Code CLI / Agent SDK session management is a different, higher layer

`claude --resume "session-name"` and the Agent SDK's `resume=` option work because **Claude Code itself** manages conversation storage on disk / server-side — this is a layer built on top of the stateless API, not an API feature itself. See §12, "Session Management", and §37, "`fork_session`", in [01-agentic-architecture-and-orchestration.md](#) for the exam-tested mechanics.

	Anthropic API	Claude Code CLI / Agent SDK
Session storage	✗ None — you manage it	✓ Managed for you
Resume mechanism	Inject prior <code>messages[]</code> yourself	<code>--resume <name></code> / <code>resume=</code> option
<code>session_id</code> parameter	✗ Does not exist on <code>messages.create()</code>	A real concept at this layer

8. Message Roles

```
{"role": "user",      "content": "..."} # Human turn
{"role": "assistant", "content": "..."} # Claude turn
# System prompt is a separate top-level param, NOT a role in messages[]
```

9. Top-level Request Parameters

```
client.messages.create(
    model=      "claude-sonnet-4-6", # required
    max_tokens= 1024,                # required – hard ceiling on output tokens
    messages=   [...],               # required – conversation history
    system=     "You are...",        # optional – system prompt
    tools=      [...],               # optional – tool definitions
    tool_choice= {...},              # optional – see section 1
    stop_sequences= ["STOP"],        # optional – custom stop strings
    stream=     True,                # optional – streaming mode
    metadata=   {"user_id": "123"},  # optional – request metadata
)
```

10. Response Object Fields

```
response.id      # unique message ID
response.type    # always "message"
```

```

response.role           # always "assistant"
response.content        # list of content blocks
response.model          # model that responded
response.stop_reason    # why it stopped (see section 3)
response.stop_sequence  # which stop sequence triggered (if any)
response.usage.input_tokens # tokens consumed by input
response.usage.output_tokens # tokens consumed by output

```

11. Multi-turn Tool Loop Pattern

The core mechanic behind **Domain 1, Task Statement 1.1** — the agentic loop lifecycle.

```

messages = [{"role": "user", "content": user_input}]
response = client.messages.create(model="claude-sonnet-4-6", max_tokens=1024, tools=tools,
messages=messages)

while response.stop_reason == "tool_use":
    tool_call = next(b for b in response.content if b.type == "tool_use")
    tool_result = run_my_tool(tool_call.name, tool_call.input)

    messages.append({"role": "assistant", "content": response.content})
    messages.append({
        "role": "user",
        "content": [{"type": "tool_result", "tool_use_id": tool_call.id, "content": tool_result}]
    })
    response = client.messages.create(model="claude-sonnet-4-6", max_tokens=1024, tools=tools,
messages=messages)

final = next(b.text for b in response.content if b.type == "text")

```

Handling tool execution failures — feed errors back, don't terminate

```

try:
    tool_result = run_my_tool(tool_call.name, tool_call.input)
    result_block = {"type": "tool_result", "tool_use_id": tool_call.id, "content": tool_result}
except Exception as e:
    result_block = {"type": "tool_result", "tool_use_id": tool_call.id, "content": str(e),
"is_error": True}

```

With `is_error: true` in context, Claude can retry with different arguments, call a fallback tool, explain the failure, or decide the task is unrecoverable — model-driven reasoning over the failure, rather than the loop terminating blindly.

See §40, "Agentic Loop Anti-Patterns", in [01-agentic-architecture-and-orchestration.md](#) for the exam's specific list of wrong ways to decide when to stop this loop — this is a directly tested gap area.

13. maxTurns

Controls how many agentic loop iterations Claude can take. **Not an API parameter** — enforced by your own loop logic (or by the CLI's `--max-turns`).

```
claude --max-turns 10 "refactor all files in /src"
```

```
MAX_TURNS = 5
turn_count = 0
while True:
    if turn_count >= MAX_TURNS:
        print(f"Reached max turns ({MAX_TURNS}). Stopping.")
        break
    response = client.messages.create(model="claude-sonnet-4-6", max_tokens=1000, tools=tools,
    messages=messages)
    turn_count += 1
    if response.stop_reason == "end_turn":
        break
    if response.stop_reason == "tool_use":
        # ... execute + append, as in section 11 ...
        pass
```

	max_tokens	maxTurns
Real API parameter	✔ Yes	✘ No
Who enforces it	Anthropic's servers	Your code / the CLI
What it limits	Words per response	Loop iterations

Exam framing (Task 1.1): a fixed iteration cap is a legitimate *safety net*, but it must never be the *primary* mechanism for deciding the loop is done — that decision belongs to `stop_reason`. Relying on `maxTurns` alone to end a loop is one of the anti-patterns the exam tests directly (see section 40).

Domain 1 — Agentic Architecture & Orchestration

Exam weight: 27% — the largest domain. Covers Task Statements 1.1–1.7: agentic loop control flow, coordinator/subagent orchestration, subagent invocation & context passing, multi-step workflow enforcement/handoff, Agent SDK hooks, task decomposition strategy, and session state/resumption/forking.

Running example used throughout: *Scenario 1 — Customer Support Resolution Agent* (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`; target 80%+ first-contact resolution) and *Scenario 3 — Multi-Agent Research System* (coordinator + `web-searcher`, `document-analyzer`, `synthesizer`, `report-writer` subagents).

40. Agentic Loop Anti-Patterns — What NOT to Check for Termination

Task Statement 1.1. The loop control flow itself is simple (§11 of [00-foundations](#)): continue while `stop_reason == "tool_use"`, stop on `"end_turn"`. What the exam actually probes is which *wrong* signals candidates reach for instead.

The three anti-patterns

Anti-pattern	Why it's wrong	What to do instead
Parsing assistant text for natural-language completion signals (e.g. scanning for "I'm done" / "Here is the final answer")	Text is unstructured and unreliable — Claude can phrase completion arbitrarily; false positives/negatives both occur	Check <code>stop_reason == "end_turn"</code> — a structured, guaranteed field
Using a fixed iteration cap (<code>maxTurns</code>) as the primary stop signal	Terminates correctly-progressing work early, or lets truly stuck loops burn all turns before anyone notices	Use <code>stop_reason</code> as the primary signal; <code>maxTurns</code> is only a safety-net ceiling around it (§13)
Checking for the presence of assistant text content as a completion indicator	Claude can emit brief text <i>alongside</i> a <code>tool_use</code> block on the same turn (e.g. "Let me check that.") — text presence does not imply the turn is final	<code>stop_reason</code> is the only structurally reliable signal; text presence/absence is not

```
# ❌ Wrong – anti-pattern: text-content check
def is_done_wrong(response):
    return any(b.type == "text" and len(b.text) > 0 for b in response.content)

# ❌ Wrong – anti-pattern: iteration cap as the actual stop decision
def is_done_wrong2(turn_count):
    return turn_count >= 10 # stops "correct" work early, or masks a stuck loop for 10 turns

# ✅ Correct – the only structurally reliable signal
def is_done(response):
    return response.stop_reason == "end_turn"
```

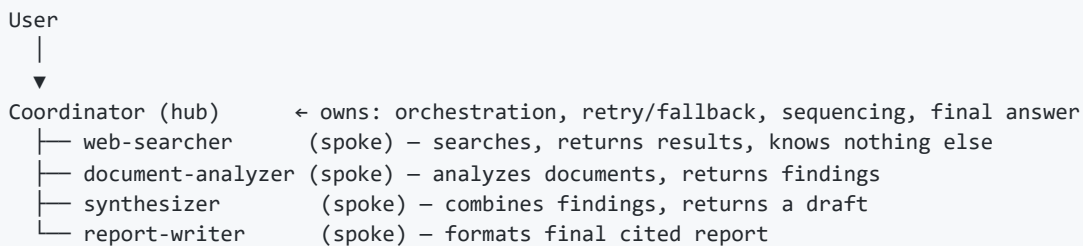
Model-driven vs. pre-configured decision trees

The exam distinguishes **model-driven decision-making** (Claude reasons about which tool to call next, given the current context) from a **pre-configured decision tree or fixed tool sequence** coded outside the model. The agentic loop pattern is model-driven by design: your code's job is to execute whatever tool Claude requests and feed the result back — not to pre-script which tool runs at step *N*. Hardcoding "always call `get_customer`, then always call `lookup_order`" defeats the purpose of an agentic loop and turns it into a scripted pipeline (see §38 for when a fixed pipeline is actually the right choice).

24. Multi-Agent Architecture — Hub-and-Spoke Pattern

Task Statement 1.2.

What it is



The coordinator manages **all** inter-subagent communication, error handling, and information routing. Subagents never talk to each other directly — everything routes through the hub. This is Anthropic's documented default multi-agent pattern.

Coordinator responsibilities

- Analyze query complexity and **dynamically decide which subagents to invoke** — not always the full pipeline (a simple factual query may only need `web-searcher`)
- Partition scope across subagents to minimize duplication (assign distinct subtopics/source types per agent, not the same broad query to all)
- Dispatch tasks, receive results (including errors), decide retry/fallback/synthesis
- Produce the final answer

The narrow-decomposition failure mode (exam sample question)

If the coordinator decomposes "impact of AI on creative industries" into only `["AI in digital art", "AI in graphic design", "AI in photography"]`, every subagent can execute flawlessly and the *system* still fails — music, writing, and film are silently missing from the report. Each subagent worked correctly within its assigned scope; the root cause is upstream, in the coordinator's task decomposition, not in any spoke. This is the single most exam-tested failure mode in this domain: **when subagents each report success but a system output has a coverage gap, check the coordinator's decomposition logs before blaming any subagent.**

Iterative refinement loops

The coordinator should evaluate the synthesis output for gaps, re-delegate to search/analysis subagents with *targeted* follow-up queries, and re-invoke synthesis — rather than treating a single decomposition pass as final.

```
draft = synthesize(findings)
gaps = coordinator_evaluate_coverage(draft, original_topic) # e.g. "no coverage of
music/writing/film"
while gaps:
    targeted_findings = [invoke_subagent("web-searcher", f"AI impact on {gap}") for gap in gaps]
    findings.extend(targeted_findings)
    draft = synthesize(findings)
    gaps = coordinator_evaluate_coverage(draft, original_topic)
```

Parallel subagent execution

Emit multiple subagent invocations **in a single coordinator response/turn**, not across separate sequential turns, to get real latency improvement:

```
# Coordinator emits both Task-tool calls in the same turn → run concurrently
# (see §36 for the Task tool / AgentDefinition mechanics)
results = await asyncio.gather(
    invoke_subagent("web-searcher", "AI impact on the music industry"),
    invoke_subagent("document-analyzer", uploaded_papers),
)
```

Error handling — coordinator owns retry/fallback, never each spoke

Situation	What the subagent reports	What the coordinator does
Transient failure, resolved internally by the subagent (e.g. a DB reconnect succeeds on retry #3)	✅ Success only — no retry history, no partial-result flag	Nothing extra — treats it as a normal success
Unrecoverable failure (timeout exhausted, hard error)	Structured error context: failure type, attempted query, partial results, alternatives tried	Retry the subagent / invoke a fallback / proceed with partial results and annotate the gap

```
results, errors = {}, {}
for name, result in subagent_outputs.items():
    if result.is_error:
        errors[name] = result.error_context          # capture what failed and why
    else:
        results[name] = result.content                # keep completed work

report = synthesize(results)
for name, ctx in errors.items():
    report.add_coverage_gap(name, reason=ctx)         # annotate, don't fabricate, don't abort
```

Do **not** put retry/fallback *decision* logic inside each subagent — that duplicates logic across every spoke and breaks the centralized control that is the defining benefit of hub-and-spoke. (Full detail on structured error propagation: §59, "Error Propagation Strategies Across Multi-Agent Systems", in [05-context-management-and-reliability.md](#).)

Pattern comparison (other topologies, for contrast)

Pattern	Central coordinator?	Best for
Hub-and-Spoke	✅ Yes	General orchestration — Anthropic's default
Hierarchical (tree)	✅ Multi-level	Very large, structured tasks with natural sub-domains
MapReduce / parallel fan-out	✅ Yes	Same task across many data chunks
Sequential Pipeline	❌ No	Linear, ordered, predictable workflows (see §38)
Peer-to-Peer / Mesh	❌ No	Rarely used — hard to debug, context isolation breaks down
Adversarial / Debate	✅ Judge	Red-teaming, verification

36. Subagent Spawning — the `Task / Agent` Tool and `AgentDefinition` (Agent SDK)

Task Statement 1.3. Verified against `code.claude.com/docs/en/agent-sdk`, including a second independent verification pass.

The `Task` tool (exam terminology) — now named `Agent` in current tooling

The exam guide's own language for this Task Statement is explicit: "*The Task tool as the mechanism for spawning subagents, and the requirement that allowedTools must include 'Task' for a coordinator to invoke subagents.*" Answer exam questions using that framing — "`Task`" is the name to recognize.

Current-product accuracy note (the tool has since been renamed): in the coordinator's `tool_use` blocks, the current Claude Code / Agent SDK tool for spawning subagents is named `Agent`, not `Task` — current code examples register "`Agent`" in `allowed_tools / allowedTools`. `Task` is the *legacy* name (Claude Code versions before v2.1.63 used it in `tool_use` blocks, and it still appears as the tool's name in the `system:init` tools list for backward compatibility). Also, listing the tool in `allowed_tools` is not a strict prerequisite the way this reference previously stated — `allowed_tools` governs *auto-approval* (skipping a permission prompt), and the subagent-spawning tool (`Agent / Task`) is documented as one of the tools that runs without asking even when not explicitly listed. Treat "allowedTools must include Task" as the exam's expected knowledge point, and "Agent, not a hard allowlist requirement" as the current-tooling reality if you're building against the SDK today.

```
from claude_agent_sdk import query, ClaudeAgentOptions, AgentDefinition

options = ClaudeAgentOptions(
    allowed_tools=["Agent", "Read", "Grep"],  # current tool name; "Task" is the exam/legacy name
    for the same mechanism
    agents={
        "web-searcher": AgentDefinition(
            description="Searches the web for a given subtopic and returns dated, sourced
findings.",
            prompt="You are a web-research subagent. Search only for the assigned subtopic. "
                "Return each finding as {claim, evidence_excerpt, source_url, published_date}.",
            tools=["WebSearch"],
        ),
        "document-analyzer": AgentDefinition(
            description="Analyzes uploaded documents for claims relevant to the research topic.",
            prompt="Extract claims with excerpt, document name, page/section, and any stated date.",
            tools=["Read", "Grep"],
        ),
    },
)

async for message in query(prompt="Research the impact of AI on creative industries",
options=options):
    ...
```

AgentDefinition fields

Field	Required?	Purpose
description	✓	What this subagent does — the coordinator model uses this to decide <i>when</i> to invoke it
prompt	✓	The subagent's system prompt / instructions
tools	optional	Restricts this subagent to a named subset (see §43, "Distributing Tools Appropriately Across Agents", in 02-tool-design-and-mcp-integration.md)
model	optional	Override the model for this subagent (e.g. a cheaper model for a narrow task)
disallowedTools / skills / mcpServers / maxTurns / permissionMode	optional	Further scoping per subagent type

Subagents do not automatically inherit parent context

A subagent invoked via `Agent` (called `Task` in the exam guide and in older tooling) starts with an **isolated context** — it does not automatically receive the coordinator's conversation history or the outputs of other subagents. Anything a subagent needs must be **explicitly included in its prompt/task description**:

```
# ✗ Wrong – assumes the synthesis subagent can see prior subagents' raw output
Agent(agent="synthesizer", prompt="Now synthesize the findings.")

# ✓ Correct – findings are explicitly embedded in the prompt
Agent(
  agent="synthesizer",
  prompt=(
    "Synthesize the following findings into a cited report.\n\n"
    f"Web search findings:\n{json.dumps(web_findings, indent=2)}\n\n"
    f"Document analysis findings:\n{json.dumps(doc_findings, indent=2)}"
  ),
)
```

Structured data over prose — preserving attribution across the handoff

Pass findings as structured objects (claim, evidence excerpt, source URL/document name, page number, date) rather than freeform prose summaries, so the receiving subagent can preserve attribution instead of re-deriving or losing it:

```
finding = {
  "claim": "Generative tools reduced average concept-art turnaround by ~40%",
  "evidence_excerpt": "...average time-to-first-draft fell from 6 days to 3.5...",
  "source_url": "https://example.org/report-2026",
  "published_date": "2026-02-14",
}
```

Fork-based session management for divergent exploration

See §37, " `fork_session` — Branching a Session (Agent SDK)", below — a related but distinct mechanism for branching a *session* (not spawning a subagent) from a shared baseline.

39. Multi-Step Workflows — Enforcement and Handoff Patterns

Task Statement 1.4.

Programmatic enforcement vs. prompt-based guidance

	Prompt-based guidance	Programmatic enforcement
Mechanism	System-prompt instruction ("always verify identity first")	A hook or gate that blocks the tool call outright
Compliance	Probabilistic — has a non-zero failure rate	Deterministic — cannot be bypassed
Use when	Ordering preference, non-critical	Financial/compliance-critical ordering (e.g. identity verification before any refund)

Exam sample: an agent that skips `get_customer` in 12% of cases and calls `lookup_order` on a stated name alone, occasionally misidentifying accounts. The fix is **not** a stronger system-prompt instruction or few-shot examples — both are prompt-based and leave a non-zero failure rate on a case with financial consequences. The fix is a **programmatic prerequisite gate** (implemented as a `PreToolUse` hook — see §21) that blocks `lookup_order` / `process_refund` until `get_customer` has returned a verified customer ID in this turn's context.

```
# .claude/settings.json — PreToolUse hook enforcing verification order
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "lookup_order|process_refund",
        "hooks": [{"type": "command", "command": "python3 scripts/require_verified_customer.py"}]
      }
    ]
  }
}
```

`require_verified_customer.py` checks whether a verified `customer_id` already exists in this session's state (e.g. written by a prior successful `get_customer` call) and exits non-zero (deny) if not — see §21 for the exact hook return-shape mechanics.

Decomposing multi-concern requests

A single customer message can carry multiple distinct issues (e.g. "my order #4821 never arrived, *and* I was double-charged"). Decompose into distinct items, investigate each **in parallel** using shared context, then synthesize one unified resolution — rather than resolving the first-mentioned issue and dropping the second, or running them as two disconnected turns that lose shared context (e.g. the same verified `customer_id`).

Structured handoff summaries for human escalation

A human agent picking up an escalation typically has **no access to the conversation transcript**. The handoff must be self-contained:

```
handoff = {
  "customer_id": "CUST-88213",
  "root_cause": "Refund policy is silent on competitor price-matching; customer requests a match to a competitor's current lower price.",
  "amount_in_question": 42.50,
  "attempted_resolution": "Offered standard 10% loyalty discount; customer declined, reiterated price-match request.",
  "recommended_action": "Escalate to pricing team – policy gap, not a customer-service judgment call.",
}
escalate_to_human(**handoff)
```

Compare with §58, "Escalation Triggers and Ambiguity Resolution", in [05-context-management-and-reliability.md](#) for *when* to escalate — this section covers *what* to hand off once the decision is made.

21. Agent SDK / Claude Code Hooks — Tool Call Interception and Data Normalization

Task Statement 1.5. Hooks are the mechanism for **deterministic** guarantees where a system prompt instruction would only be **probabilistic**.

Hook types

Hook	Runs	Can block?	Use for
PreToolUse	Before the tool executes	✔ Yes — set a deny decision	Enforcing preconditions, blocking policy-violating calls
PostToolUse	After the tool executes	✘ No — tool already ran	Normalizing heterogeneous result data, logging, side effects
Notification	On status/notification events	✘ No	Alerting
Stop	When Claude finishes a turn	✘ No	Summaries, notifications

A `Notification` hook cannot enforce ordering: the tool **executes before** the notification fires, so a warning logged in a `Notification` hook means the refund already ran. Only `PreToolUse` can prevent execution.

`PreToolUse` — blocking a policy-violating action (Scenario 1: refund threshold)

```
def pre_tool_hook(tool_name, tool_input):
    if tool_name == "process_refund":
        amount = tool_input["amount"]
        if isinstance(amount, str):
            amount = float(amount.replace("$", "").replace(",", "")) # normalize + enforce
```

```

together

if amount > 500:
    return {"hookSpecificOutput": {
        "hookEventName": "PreToolUse",
        "permissionDecision": "deny",
        "permissionReason": f"${amount} exceeds the $500 auto-approval threshold",
    }}
    # caller/coordinator should redirect this case to escalate_to_human

return {"hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "allow",
    "updatedInput": {**tool_input, "amount": amount},
}}

```

Return shape: a `hookSpecificOutput` object with `hookEventName`, `permissionDecision` ("allow" / "deny" / "ask" — there is no "defer"), an optional `permissionReason`, and — only on `PreToolUse` — an optional `updatedInput` to rewrite the tool call before it runs.

Rule: if a normalize step and an enforce step must see each other's output, put them in **one** hook — `updatedInput` from one `PreToolUse` hook does **not** propagate to a second `PreToolUse` hook; each hook receives the *original* input.

PostToolUse — normalizing heterogeneous MCP tool output (Scenario 1 / 3)

Different backend tools return dates as Unix timestamps, ISO 8601 strings, or numeric status codes inconsistently. A `PostToolUse` hook normalizes this **before** the model reasons over it, so the model never has to hold multiple date formats in its head:

```

def post_tool_hook(tool_name, tool_output):
    if tool_name == "lookup_order":
        tool_output["order_date"] = normalize_to_iso8601(tool_output["order_date"])
        tool_output["status"] = STATUS_CODE_MAP.get(tool_output["status_code"], "unknown")
    return tool_output

```

Multiple `PreToolUse` hooks on the same event — most-restrictive wins

```
deny > ask > allow > {} (abstain)
```

If an identity-verification hook returns `deny` and a fraud-score hook returns `allow` for the same call, the call is **blocked** — `deny` from any hook overrides `allow` from every other hook. (There is no `defer` decision value — only `allow`/`deny`/`ask`.)

Deterministic (hooks) vs. probabilistic (prompt) — the decision rule

Requirement	Use
"Always verify identity before a refund, no exceptions"	<code>PreToolUse</code> hook (hard gate)
"Prefer a friendly tone"	System prompt (guidance is fine — no compliance stakes)

Requirement	Use
"Never approve a refund over \$500 without a human"	<code>PreToolUse</code> hook (financial/compliance-critical)
"Generally try <code>get_customer</code> before <code>lookup_order</code> "	Prompt/few-shot <i>if</i> the cost of an occasional miss is low; a hook if it isn't (see §39)

See §46/§47, "Choosing the Right Claude Code Configuration Mechanism" / "Enforcement Layer vs. Guidance Layer", in [03-claude-code-configuration-and-workflows.md](#) for the broader decision framework across CLAUDE.md / rules / skills / hooks / permissions — one of the most heavily tested distinctions in Domain 3, and the same "deterministic vs. probabilistic" logic drives both.

38. Task Decomposition Strategies for Complex Workflows

Task Statement 1.6.

Fixed pipeline (prompt chaining) vs. dynamic adaptive decomposition

	Fixed pipeline (prompt chaining)	Dynamic adaptive decomposition
Subtasks	Predetermined, sequential, known in advance	Generated based on what is discovered at each step
Best for	Predictable, multi-aspect reviews (e.g. "analyze each file, then run a cross-file pass")	Open-ended investigation where scope isn't knowable upfront
Failure mode if misapplied	Using a fixed pipeline for open-ended work misses discoveries the plan didn't anticipate	Using full dynamic decomposition for a predictable review adds needless coordination overhead

Prompt chaining example — large code review (Scenario 5: CI)

Step 1: for each changed file → local analysis pass (bugs, style, missing tests, in isolation)
Step 2: one cross-file integration pass → data flow between files, contradictory findings, duplicated logic

Splitting this way avoids **attention dilution**: a single pass across 14 changed files produces inconsistent depth (detailed feedback for some files, superficial for others) and can flag the same pattern as a problem in one file while approving it in another. Per-file passes give consistent depth; the separate integration pass is where cross-file issues belong — merging both concerns into one pass is what caused the dilution in the first place.

Dynamic decomposition example — open-ended task (Scenario 4: dev productivity)

"Add comprehensive tests to a legacy codebase" cannot be decomposed upfront — you don't yet know which modules are undertested or which are highest-risk. The adaptive approach:

```
structure = explore_agent.map_codebase_structure()           # phase 1: what exists
high_impact = explore_agent.identify_high_impact_areas(structure) # phase 2: what matters, informed
```

```
by phase 1
plan = build_prioritized_test_plan(high_impact)          # phase 3: adapts as dependencies are
discovered
```

Each phase's subtasks are generated from the *previous* phase's findings — not fixed at the start. This is the same principle as §24's narrow-decomposition failure: a plan fixed too early (before discovery) systematically under-covers the true scope.

Decision rule

- Scope and steps are **knowable in advance**, and depth/consistency matter more than adaptability → **prompt chaining**
- Scope is **discovered as you go**, and a fixed plan would systematically miss things → **dynamic decomposition**

12. Session Management (Claude Code CLI and Agent SDK)

Task Statement 1.7 (part 1).

Named sessions

```
claude --name "migration-task"      # start a named session
/rename "migration-task"            # rename during a session
claude --resume "migration-task"    # resume by name
claude --resume                      # no argument → interactive picker
```

Resuming by explicit `session_id` — the CI/parallel-job pattern

When multiple `claude -p` invocations run concurrently (e.g. parallel CI jobs reviewing different PRs — Scenario 5), `--continue` is unreliable because it attaches to the *most recent* session by timestamp, which may belong to a different concurrent job. Capture `session_id` from the first call's JSON output and pass it explicitly:

```
RESPONSE=$(git diff origin/main | claude -p 'review this PR for bugs' --output-format json)
SESSION_ID=$(echo "$RESPONSE" | jq -r '.session_id')
claude -p 'what is the most critical issue you found?' --resume "$SESSION_ID"
```

Informing a resumed session about file changes

When resuming a session after the underlying code has changed, tell the agent explicitly which files changed rather than assuming it will re-detect drift — stale tool results (a file read before the edit) otherwise silently persist in context. For targeted re-analysis: `claude --resume "session-name" "Files auth.py and db/pool.py changed since we last spoke – re-check just those against your earlier findings."`

Resume vs. fresh session with an injected summary

Situation	Use
Prior context is still mostly valid, just continuing the same investigation	<code>--resume</code>
Prior tool results are now stale (files changed, external state moved on)	Start fresh, inject a structured summary of what's still true, rather than resuming and hoping the model reconciles stale reads itself

37. `fork_session` — Branching a Session (Agent SDK)

Task Statement 1.7 (part 2). *Verified against `code.claude.com/docs/en/agent-sdk/sessions`.*

`fork_session` (Python) / `forkSession` (TypeScript) is an option passed alongside `resume` to `query()`. It creates a **new, independent session** that starts as a copy of the resumed session's history — giving it its own new session ID — rather than continuing to mutate the original session:

```
from claude_agent_sdk import query, ClaudeAgentOptions

# Baseline: shared codebase analysis already happened in session `baseline_id`

# Branch A — explore a REST-based refactor
async for msg in query(
    prompt="Refactor this using a REST API layered architecture.",
    options=ClaudeAgentOptions(resume=baseline_id, fork_session=True),
):
    ...

# Branch B — explore an event-driven refactor, from the SAME baseline, independently
async for msg in query(
    prompt="Refactor this using an event-driven architecture instead.",
    options=ClaudeAgentOptions(resume=baseline_id, fork_session=True),
):
    ...
```

Both branches see the same shared analysis baseline but diverge independently from that point — exploring Branch A does not affect Branch B, and neither mutates the original `baseline_id` session. This is the correct mechanism for "compare two approaches from the same starting analysis" (e.g. two testing strategies, two refactoring directions), as distinct from `--resume` (continues the *same* session in place) and from spawning a Task subagent (isolated context, not a branch of *this* conversation's history).

Mechanism	Continues same session?	New independent session created?	Shares prior history?
<code>--resume</code> / <code>resume=</code> (no fork)	✔ Yes, in place	✗ No	✔ Yes — same session
<code>resume=</code> + <code>fork_session=True</code>	✗ No	✔ Yes	✔ Yes — copied at fork point, then diverges
Task subagent spawn (§36)	N/A — not a session concept	N/A	✗ No — isolated context, explicit prompt only

Domain 2 — Tool Design & MCP Integration

Exam weight: 18%. Covers Task Statements 2.1–2.5: tool interface design, structured MCP error responses, distributing tools across agents & `tool_choice`, integrating MCP servers into Claude Code, and selecting built-in tools (Read/Write/Edit/Bash/Grep/Glob) effectively.

Running example used throughout: *Scenario 1 — Customer Support Resolution Agent* (`get_customer` vs `lookup_order` disambiguation) and *Scenario 4 — Developer Productivity* (MCP server integration, built-in tool selection while exploring an unfamiliar codebase).

41. Tool Description Quality and Disambiguation

Task Statement 2.1.

Descriptions are the primary selection mechanism

An LLM chooses which tool to call almost entirely from its `description` field (plus the surrounding system prompt). A minimal description — "Retrieves customer information" / "Retrieves order details" — gives the model no basis to differentiate two similarly-shaped tools, and misrouting follows: a query like "check my order #12345" gets routed to `get_customer` because nothing in either description says which tool owns order-shaped identifiers.

The fix, in priority order

1. **Expand the descriptions first** — input formats it handles, example queries, edge cases, and explicit boundaries versus the similar tool. This is the correct **first step**: low effort, directly addresses the root cause (the model lacks disambiguating information), and requires no new infrastructure.
2. Only if descriptions alone don't resolve ambiguity, consider renaming for clarity or splitting an overloaded tool into purpose-specific ones (below) — higher-effort structural changes, not first-line fixes.

```
# ❌ Before — indistinguishable to the model
{"name": "get_customer", "description": "Retrieves customer information"}
{"name": "lookup_order", "description": "Retrieves order details"}

# ✅ After — explicit boundaries, formats, and disambiguation
{
  "name": "get_customer",
  "description": (
    "Look up a customer record by email or phone number ONLY. "
    "Returns customer_id, name, verification status. "
    "Do NOT use this for order-related queries (order numbers, shipment status) – "
    "use lookup_order for anything that includes an order number like '#12345'."
  ),
}
{
  "name": "lookup_order",
  "description": (
    "Look up an order by its order number (format: '#' followed by 4-6 digits, e.g. '#12345'). "
    "Returns order status, items, shipment tracking. "
    "Requires a verified customer_id from get_customer first – call get_customer before this "
    "if the customer has not yet been verified in this conversation."
  )
}
```

```
    },  
  }
```

Why the other common fixes are wrong *as a first step*

Alternative fix	Why it's the wrong first move
Few-shot examples showing correct routing	Adds token overhead on every request without fixing the underlying ambiguity in the tool contracts themselves
A routing layer that pre-parses input and pre-selects a tool by keyword	Over-engineered; bypasses the model's semantic understanding with brittle keyword matching
Consolidate both tools into one generic <code>lookup_entity</code> tool	A legitimate structural option (see below), but higher-effort than warranted when the root cause is simply thin descriptions

Splitting an overloaded tool (when descriptions alone can't fix it)

If a tool is *genuinely* trying to do two things (e.g. `analyze_document` doing extraction, summarization, *and* verification), splitting into purpose-specific tools with distinct input/output contracts removes the ambiguity structurally:

```
# Before: one overloaded tool, ambiguous by design  
{  
  "name": "analyze_document",  
  "description": "Analyzes a document"  
}  
  
# After: three tools, each with a defined contract  
{  
  "name": "extract_data_points",  
  "description": "Extracts named fields (dates, amounts, parties) from a document into structured JSON."  
},  
{  
  "name": "summarize_content",  
  "description": "Produces a prose summary of a document's content."  
},  
{  
  "name": "verify_claim_against_source",  
  "description": "Checks whether a specific claim is supported by a given document, returning true/false + citation."  
}
```

System-prompt keyword sensitivity

Wording in the system prompt can create unintended tool associations independent of the tool descriptions themselves — e.g. a system prompt that repeatedly says "search for information" can bias the model toward a `web_search`-like tool even when a more specific tool is the correct choice. Review system-prompt wording for keyword overlap with tool names/descriptions when debugging unexpected tool selection.

42. Structured Error Responses for MCP Tools

Task Statement 2.2.

The `isError` flag

MCP tool results carry a boolean `isError` flag distinguishing success from failure at the protocol level. A uniform, generic error string (`"Operation failed"`) on every failure gives the agent no basis for a recovery decision — it cannot tell a retryable timeout from a permanent policy violation from a permission problem.

Error categories and structured metadata

Category	Example	isRetryable	Agent's correct response
Transient	Timeout, service temporarily unavailable	true	Retry (possibly with backoff)
Validation	Malformed input (bad date format, missing field)	true (after correcting input)	Correct the input and retry
Business	Refund exceeds policy threshold	false	Explain to the user / route to an alternative workflow — retrying changes nothing
Permission	Agent lacks scope for this action	false	Escalate to a human or a higher-privilege path — retrying changes nothing

```
# Structured MCP error result — every field is actionable, not just human-readable
{
  "isError": True,
  "errorCategory": "business",
  "isRetryable": False,
  "message": "Refund amount $650 exceeds the $500 auto-approval threshold for this account tier.",
  "suggestedNextAction": "escalate_to_human",
}
```

Why this matters for agent behavior

Without `errorCategory` / `isRetryable`, an agent facing any failure has two bad defaults: retry blindly (wastes calls on a business-rule rejection that will never succeed) or give up immediately (abandons a transient failure that would have succeeded on retry). Structured metadata lets the agent's own reasoning make the correct choice per failure type.

Access failures vs. valid empty results

A tool call that **fails** (timeout, connection error) is not the same as a tool call that **succeeds with no matches** (a valid, successful query that legitimately found nothing). Conflating the two — e.g. returning an empty list for both — hides a real failure behind what looks like a normal "no results" case. Distinguish them explicitly:

```
# Access failure — needs a retry/fallback decision
{"isError": True, "errorCategory": "transient", "isRetryable": True, "message": "Order service timed out after 3 attempts."}

# Valid empty result — success, nothing to retry
{"isError": False, "results": [], "message": "Query succeeded; no orders found for this customer."}
```

Local recovery before escalation (subagent context)

A subagent should attempt local recovery for transient failures itself (e.g. retry a flaky connection 2–3 times) and only propagate to the coordinator the errors it **cannot** resolve — along with what was attempted and any partial results obtained. Reporting a resolved transient failure upward as if it were still a problem, or reporting it with `is_error: true` retry history attached, misrepresents a task that actually succeeded. See

\$59, "Error Propagation Strategies Across Multi-Agent Systems", in [05-context-management-and-reliability.md](#) for the coordinator-side handling of what a subagent does escalate.

43. Distributing Tools Appropriately Across Agents

Task Statement 2.3.

Too many tools degrades selection reliability

Giving one agent access to 18 tools instead of 4–5 increases decision complexity and measurably degrades tool-selection accuracy — even when each individual tool description is well written. The fix is architectural, not prompt-level: **scope each agent to only the tools its role needs.**

```
# ❌ One agent, every tool – degrades reliability
support_agent_tools = [get_customer, lookup_order, process_refund, escalate_to_human,
                      web_search, send_email, update_inventory, generate_invoice,
                      void_transaction, apply_credit, ... ] # 18 tools

# ✅ Scoped per role (Scenario 1)
support_agent_tools = [get_customer, lookup_order, process_refund, escalate_to_human] # 4 tools
```

Agents given tools outside their specialization tend to misuse them — e.g. a `synthesizer` subagent (Scenario 3) given web-search access will sometimes attempt its own searches instead of relying on the `web-searcher` subagent's findings, duplicating work and bypassing the coordinator's routing.

Scoped cross-role tools for high-frequency needs

When a subagent needs *some* capability outside its core role frequently enough that always routing through the coordinator adds meaningful latency, give it a narrow, purpose-built tool for that specific need — not full access to the other role's tools:

```
# synthesizer needs to fact-check simple claims often (85% of cases are simple lookups)
# Give it ONE scoped tool, not the full web-searcher toolset:
{"name": "verify_fact", "description": "Checks a single factual claim (date, name, statistic)
against a quick web lookup. For deep research, this is NOT a substitute for the web-searcher
subagent."}
```

Route the remaining 15% (deeper investigation) through the coordinator → `web-searcher`, as before. This is least-privilege applied to the common case, without over-provisioning the subagent for the rare case.

`tool_choice` as an access-scoping mechanism per turn

Beyond which tools an agent *has*, `tool_choice` controls which of them can fire *on a given turn* — see §1, "Tool Choice", in [00-foundations-messages-api.md](#) for the full `auto / any / tool / none` reference. Forcing a specific tool (`{"type": "tool", "name": "extract_metadata"}`) is the mechanism for enforcing a pipeline's first step; `"any"` guarantees some tool fires when a plain-text fallback is unacceptable (Domain 4 territory).

44. Configuring MCP Servers at the Correct Scope

Task Statement 2.4.

Project-level vs. user-level — choosing where a server config lives

Scope	File	Use for	Shared with team?
Project	<code>.mcp.json</code> at repo root	Servers the whole team needs — shared backend tools, a company Jira/GitHub MCP server	✅ Yes — committed to version control
User	<code>~/.claude.json</code> (user-level config)	Personal or experimental servers — a server you're evaluating, a personal productivity integration	❌ No — stays on your machine

The decision rule is **who else needs it, and is it stable enough to commit**: a server every teammate must have to work in this repo → project scope; a server that's yours alone, or still being evaluated → user scope. Configuring a stable, shared tool at user scope means new team members silently lack it; configuring an experimental/personal server at project scope pushes half-finished or personal config onto every teammate.

```
// .mcp.json (project root, committed) – shared team tooling
{
  "mcpServers": {
    "jira": {
      "command": "npx",
      "args": ["-y", "@company/jira-mcp-server"],
      "env": { "JIRA_API_TOKEN": "${JIRA_API_TOKEN}" } // env var expansion – no secret committed
    }
  }
}
```

```
// ~/.claude.json (user-level, not committed) – personal/experimental
{
  "mcpServers": {
    "my-experimental-notes-server": {
      "command": "node", "args": ["/Users/me/dev/notes-mcp/index.js"]
    }
  }
}
```

Both scopes are discovered and available simultaneously

Tools from every configured MCP server — project *and* user scope — are discovered at connection time and available to the agent in the same session; the agent doesn't need to explicitly "switch on" either scope.

Scope precedence for duplicate server names

When the **same server name** is defined at more than one scope, Claude Code does not merge fields — it uses the entire definition from the single **highest-precedence** scope:

```
managed (org-provided) > local > project > user > plugin-provided > claude.ai connectors
```

A developer's personal **local**-scoped override of a shared **project**-scoped `analytics` server wins — `local` always outranks `project`, regardless of which one is version-controlled. This is expected, resolved-automatically behavior, not a configuration error. (`managedMcpServers` is an org-provided tier that outranks even `local`, available in newer Claude Code versions — not the exam's focus, but good to know it exists if you see it in current tooling.)

Environment variable expansion in `.mcp.json`

```
{"args": ["--region", "${API_REGION:-us-east-1}"], "env": {"API_KEY": "${MY_API_KEY}"}}
```

`${VAR:-default}` falls back to the literal default if `VAR` is unset. A bare `${VAR}` with no default, left unset, does **not** silently blank out — Claude Code emits a missing-variable warning and loads the server using the **literal, unexpanded** `${VAR}` text, which will typically fail downstream (e.g. as a bad argument or credential) rather than failing at config-parse time. This mechanic is config-wide — it works identically in `command`, `args`, `env`, `url`, and `headers` — which is what lets one `.mcp.json` be committed and shared while still tolerating machine-specific values, as long as a sensible default is supplied for anything optional.

MCP resources — reducing exploratory tool calls

MCP servers expose **resources** (readable content catalogs — issue summaries, documentation hierarchies, database schemas) distinct from **tools** (callable actions). Referencing a resource inline with `@server:protocol://path` gives the agent visibility into available data without needing a preliminary "list what's available" tool call:

```
@docs:file://api/authentication
```

Prefer existing community servers for standard integrations

For standard integrations (Jira, GitHub, Slack), prefer an existing, maintained community MCP server over building a custom one — reserve custom server development for genuinely team-specific workflows that no general-purpose server covers.

45. Systematic Codebase Exploration — Grep, Glob, Read Together

Task Statement 2.5.

Building understanding incrementally, not exhaustively

The efficient pattern for exploring an unfamiliar codebase (Scenario 4) is: **Grep to find entry points → Read to follow imports and trace flows → repeat, narrowing** — not reading every file upfront. Reading the entire codebase before answering any question burns context on files that turn out to be irrelevant to the actual task, and it does not scale as the codebase grows.

```
# Step 1 – Grep to locate the concept, not read everything
Grep(pattern="class OrderService", type="py")      # → finds definition location

# Step 2 – Read only the file(s) Grep pointed at
Read(file_path="src/services/order/OrderService.py")  # → follow imports/inheritance from here

# Step 3 – Grep again for the next concept surfaced by Step 2 (e.g. a base class), Read that
Grep(pattern="class TransactionalBase", type="py")
Read(file_path="src/services/base/TransactionalBase.py")
```

Choosing the right tool for the question

Goal	Right tool	Wrong tool (and why)
Find files by name/extension pattern	Glob (<code>**/*.test.tsx</code>)	Grep — searches content, not paths; misses files whose content never contains the search term
Search file contents for a symbol, error message, or import	Grep	Glob — cannot see inside files
Trace a function's usage across wrapper modules	Grep for each exported name, after identifying all export names first	Guessing one name and missing re-exports
Read a specific file fully	Read	Bash (<code>cat ...</code>) — unnecessary shell overhead
Run a test suite and see pass/fail + stack trace	Bash	Read / Glob / Grep — none of them execute anything

Managing context window constraints while exploring

Incremental exploration (above) is itself a context-management technique — reading only what each step's findings point to keeps far less irrelevant content in the window than an upfront full-repo read. For sessions long enough that this still isn't sufficient (multi-hour codebase exploration), combine with the scratchpad-file and subagent-delegation patterns in §60, "Context Management in Large Codebase Exploration", in [05-context-management-and-reliability.md](#).

23. Built-in Tool Selection — Bash vs. Read / Glob / Grep, and Edit vs. Write

Task Statement 2.5 (continued). The rule: **static file operations use dedicated tools; execution and live output require Bash.**

Goal	Correct tool	Wrong tool (and why it fails)
Run a test suite and capture a stack trace	Bash	Read reads config, not live output; Glob finds files, not pass/fail status
Find files by name pattern	Glob	Bash (<code>find ...</code>) works but is slower and less integrated

Goal	Correct tool	Wrong tool (and why it fails)
Search file contents for a symbol	Grep	Bash(grep ...) works but bypasses the optimized tool
Read a specific file's contents	Read	Bash(cat ...) — unnecessary overhead
Run any terminal command (build, lint, deploy, git)	Bash	No substitute — only Bash executes commands

Edit VS. Write

Scenario	Right tool
Fix a single function name in a file	Edit (targeted, unique-text match)
Reformat indentation across a 900-line file	Write (after Read) — hundreds of fragile per-line Edit calls is the wrong approach
Edit fails because the anchor text isn't unique	Fall back to Read + Write — read the full file, transform in memory, write it back whole

```
content = Read(file_path="generated_output.py")
reformatted = reindent(content)
Write(file_path="generated_output.py", content=reformatted)
```

Domain 3 — Claude Code Configuration & Workflows

Exam weight: 20%. Covers Task Statements 3.1–3.6: CLAUDE.md hierarchy/scoping, custom slash commands & skills, path-specific rules, plan mode vs. direct execution, iterative refinement, and CI/CD integration.

Running example used throughout: *Scenario 2 — Code Generation with Claude Code* (custom `/review` command, CLAUDE.md conventions, plan mode for a monolith→microservices split) and *Scenario 5 — Claude Code for CI/CD* (automated PR review and test generation).

This domain had the weakest measured coverage on the last practice attempt — sections 46/47/48 below target the 0%-scored objectives directly.

46. Choosing the Right Claude Code Configuration Mechanism

Task Statement 3.1 / cross-cutting. Five mechanisms exist, each answering a different question. Confusing them is the single most-tested configuration mistake in this domain.

Mechanism	Answers	Loaded	Can it <i>block</i> an action?
CLAUDE.md	"What should Claude always know about this project?"	Every session, unconditionally	✗ No — guidance only
<code>.claude/rules/*.md</code> (no paths)	Same as CLAUDE.md, but organized as separate topic files	Every session, unconditionally	✗ No
<code>.claude/rules/*.md</code> (with paths:)	"What convention applies only when touching files matching X?"	Only when a matching file is read/edited	✗ No
Skills (<code>.claude/skills/</code>)	"What's an on-demand, invokable workflow?"	Only when explicitly invoked (<code>/name</code>)	✗ No (unless the skill itself uses a hook)
Hooks (<code>settings.json</code>)	"What must be true/false, guaranteed, every time?"	Fires on the matching lifecycle event	✓ Yes — <code>PreToolUse</code> can deny
permissions (<code>settings.json</code>)	"What is this session allowed to do at all?"	Every tool call, checked against the allow/deny list	✓ Yes — denies outright

Worked decision — "tests must follow one convention, regardless of directory"

Test files sit throughout the repo (`Button.test.tsx` next to `Button.tsx`, `handlers/order.test.ts` next to `order.ts`). The requirement is: apply the same testing convention automatically, based on file *type*, not directory.

- ✗ A separate CLAUDE.md per subdirectory — doesn't scale across many directories, and CLAUDE.md is directory-bound
- ✗ One root CLAUDE.md with a "testing conventions" header, relying on Claude to infer when it applies — inference is not deterministic matching
- ✗ A skill — requires manual invocation (or Claude choosing to load it); contradicts "automatic on file touch"
- ✓ `.claude/rules/testing.md` with paths: `["**/*.test.tsx", "**/*.test.ts"]` — deterministic, glob-based, directory-independent

Worked decision — "identity verification must happen before any refund, no exceptions"

- ✗ CLAUDE.md instruction "always verify identity first" — probabilistic; a documented 12% miss rate in production is exactly this failure mode (see §39 in [01-agentic-architecture-and-orchestration.md](#))
- ✗ A skill that runs the verification step — still requires the model to choose to invoke it
- ✓ `PreToolUse` **hook** blocking `process_refund` until a verified `customer_id` exists in context — deterministic, cannot be skipped

The general rule (also see §47)

If the requirement can tolerate an occasional miss, it belongs in CLAUDE.md / rules / skills (guidance). If it cannot — compliance, financial, safety-critical, "must never happen" — it belongs in a hook or a `permissions` deny rule (enforcement). Restructuring a misconfigured setup means moving the *specific instruction* to the layer whose guarantee matches its actual criticality, not adding more emphatic wording at the same layer.

47. Enforcement Layer vs. Guidance Layer — the Deterministic/Probabilistic Split

Task Statement 3.1 / 1.4 / 1.5 (cross-domain). A companion framing to §46, focused specifically on *why* prompt-based instructions have a non-zero failure rate even when worded emphatically.

Layer	Failure mode if used for the wrong requirement
CLAUDE.md / rules (guidance)	Model <i>usually</i> complies but occasionally doesn't — acceptable for style/preference, unacceptable for compliance
Skills	Only fires if invoked — never a substitute for "must always happen"
Hooks / permissions (enforcement)	Overkill (and adds latency/complexity) for a soft preference that doesn't need a hard guarantee

The exam frames this explicitly under Domain 1 Task 1.4/1.5 (Agent SDK hooks — see §21 and §39 in [01-agentic-architecture-and-orchestration.md](#)) **and** under Domain 3 Task 3.1 (CLAUDE.md/rules hierarchy) — it is the same underlying principle tested from two angles: *build* it right (Domain 1's hook mechanics) and *choose the right place to put it* (Domain 3's configuration-mechanism selection, §46).

48. `PostToolUse` Hooks for Automatic Code-Quality Enforcement

Task Statement 3.1 / 1.5. A concrete, fully-worked example of deterministic enforcement that does not depend on the model remembering to run linting/tests.

```
// .claude/settings.json – runs after every Edit or Write, unconditionally
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          { "type": "command", "command": "npx prettier --write $CLAUDE_FILE_PATH" },
          { "type": "command", "command": "npx eslint --fix $CLAUDE_FILE_PATH" },
          { "type": "command", "command": "npm test -- --findRelatedTests $CLAUDE_FILE_PATH" }
        ]
      }
    ]
  }
}
```

Why this must be a hook and not a CLAUDE.md instruction ("run the linter after editing files"): a model that is deep in a multi-file refactor can simply forget, skip it under time/context pressure, or judge (incorrectly) that it's optional for a given edit. A `PostToolUse` hook runs **every time**, independent of whether the model thought to ask for it — this is exactly the "independent of model instruction-following" framing the exam uses for this objective.

`PostToolUse` cannot *block* the edit (the file write already happened) — it's the right hook for *reactive* enforcement (fix formatting, re-run affected tests) as opposed to `PreToolUse`, which is for *preventing* an action before it happens (§21, §39).

16. Custom Slash Commands and Skills — Scope and Configuration

Task Statement 3.2.

Project vs. user scope

Location	Committed / shared?	Use for
<code>.claude/commands/</code> or <code>.claude/skills/<name>/SKILL.md</code>	✔ Yes — version-controlled	Team-wide workflows every developer should get on clone (Scenario 2: a <code>/review</code> command running the team's checklist)
<code>~/.claude/commands/</code> or <code>~/.claude/skills/<name>/SKILL.md</code>	✗ No — personal	Individual habits/preferences, never pushed onto teammates

Exam sample-question framing: a `/review` command meant for every developer on clone belongs in `.claude/commands/` (project-scoped, version-controlled) — not `~/.claude/commands/` (personal only), not `CLAUDE.md` (project context/instructions, not a command definition), and there is no `.claude/config.json` commands array in Claude Code.

Skills are the current best-practice format; commands are legacy

Both become a `/name` slash command. **Skills** (`SKILL.md` + YAML frontmatter) unlock configuration options that plain commands don't have:

```
---
name: pr-review-checklist
description: Runs the team's standard PR review checklist against the current diff.
argument-hint: "[severity-level]"
allowed-tools: Read, Grep, Glob
context: fork
---
Review the current diff against .claude/rules/api-conventions.md and report findings grouped by severity. If an argument is given, only report findings at or above that severity.
```

Frontmatter field	Purpose
<code>description</code>	What the skill does (also used for autocomplete / model-invocation decisions)
<code>argument-hint</code>	A hint string shown in autocomplete when the user types <code>/pr-review-checklist</code> with no arguments yet — prompts them for the expected parameter shape
<code>allowed-tools</code>	Auto-approves the listed tools for this invocation (a UX convenience, not a security sandbox — unlisted tools still work, just with a prompt)
<code>context: fork</code>	Runs the skill in an isolated subagent; only the final result returns to the main conversation — keeps verbose intermediate steps (e.g. a full codebase scan) out of the main session's context
<code>paths:</code>	Auto-activates the skill when a matching file is touched

Frontmatter field	Purpose
<code>user-invocable: false</code>	Hides the skill from the <code>/</code> menu (still usable programmatically)

`context: fork` — isolating verbose or exploratory output

Use `context: fork` for skills that produce a lot of intermediate noise (a full codebase analysis, brainstorming several competing approaches) so only the final recommendation pollutes the main conversation:

```
---
context: fork
agent: Explore
---
Map this codebase's module structure and return a one-paragraph summary.
```

Caveat: the agent type named in `agent:` (not `context: fork` itself) determines what context the forked subagent starts with. `Explore` is deliberately lean and does **not** load `CLAUDE.md` by default — a skill forked into `Explore` cannot reflect project conventions defined there. Forking into `general-purpose` instead loads `CLAUDE.md` normally.

Personal skill customization

A personal variant lives at `~/.claude/skills/<name>/SKILL.md` under a **different name** from the project version, so it doesn't shadow or conflict with the team's shared skill and never appears in the repo.

22. `.claude/rules/` and Path-Scoped Loading

Task Statement 3.3.

`paths:` **frontmatter** — conditional vs. unconditional loading

`.claude/rules/*.md` files carry the same YAML-frontmatter convention as skills. Whether the frontmatter has a `paths:` key changes **when** the rule loads:

```
---
# no `paths` key – loads at session start, every session, same priority as CLAUDE.md
---
Use 2-space indentation. Prefer named exports.
```

```
---
paths:
- "terraform/**/*"
---
```

Apply terraform-specific conventions here. Stays out of context until a matching file is read or edited.

Why glob-scoped rules beat directory-level CLAUDE.md for scattered conventions

A convention that must apply to every test file regardless of location (`Button.test.tsx` beside `Button.tsx`, `order.test.ts` beside `order.ts`, `__tests__/checkout.test.tsx`) cannot be expressed by a directory-scoped CLAUDE.md, because CLAUDE.md files apply per-directory, and tests aren't confined to one directory. A single `.claude/rules/testing.md` with `paths: [ "**/*.test.tsx", "**/*.test.ts" ]` applies uniformly regardless of where the file lives.

/memory — diagnosing instruction-loading inconsistencies across developers

`/memory` lists every CLAUDE.md, CLAUDE.local.md, and rules file actually loaded in the current session. If two engineers' outputs differ, the cause is a loading/scoping issue (wrong working directory, missing file, dead symlink, `claudeMdExcludes` — see §34 below — filtering it out); if the lists match, the problem is in the instruction wording itself, not loading.

18. Path-Scoped Rules and Symlinks in Claude Code

Task Statement 3.3 (continued).

Glob semantics — * vs **

Pattern	Matches
<code>**/*.test.tsx</code>	<code>.test.tsx</code> files at any depth — the right choice for tests scattered across the repo
<code>src/**/*.test.tsx</code>	Same, but only under <code>src/</code>
<code>src/components/*.test.tsx</code>	Only directly in <code>src/components/</code> , no subdirectories

`*` never crosses a `/` boundary; `**` does. `paths` is always a YAML **list**, even for one pattern (`paths: "x"` is invalid; `paths: [ "x" ]` is correct).

Symlinks — rules travel across repos

`.claude/rules/` supports symlinking individual files or the whole directory from an external shared source (`ln -s ~/org-shared-rules/security.md .claude/rules/security.md`), auto-updating when the source changes, with no submodule requirement. Rules also match on both the symlinked path **and** the canonical path — no extra `paths` entries are needed for a symlinked checkout.

20. Claude Code Permission Modes

Task Statement 3.1 / CI context (3.6).

Mode	Unapproved action	Human required?	Use for
default	Prompts user	✅ Yes	Normal interactive development
acceptEdits	Auto-approves file edits + basic filesystem ops; still prompts for shell commands / network requests	Partial	Dev workflows with frequent file changes
plan	Explores/plans without executing changes (see §49)	✅ Yes, to approve the plan	Architectural or multi-file work needing review before execution
dontAsk	Denies silently, no prompt	❌ No	CI/CD — deterministic, non-hanging
bypassPermissions	Allows everything	❌ No	Dangerous — sandboxed use only

There is also an `auto` mode (model-classified approvals) in current tooling, not covered by the exam guide's own permission-mode enumeration — the four above are the ones the exam explicitly tests.

`dontAsk` + an explicit `permissions.allow` list is the deterministic CI pattern: everything not on the allowlist fails fast and silently rather than hanging on a human who isn't present.

```
// .claude/settings.json — locked-down CI runner
{
  "permissionMode": "dontAsk",
  "permissions": {"allow": ["Bash(git log:*)", "Bash(git diff:*)", "Read"]}
}
```

32. Blocking Bash in Non-Interactive / CI Pipelines — `--disallowedTools`

Supporting fact for Domain 3 (CI context, Task Statement 3.6), gathered from prior Q&A.

```
git diff main | claude -p "you are a typo linter..." --disallowedTools Bash
```

The only authoritative ways to restrict tool access: `--disallowedTools <Tool>`, `--allowedTools <Tool>`, or a project-level `permissions.deny` in `.claude/settings.json`. Prompt text ("do not use Bash") is not a permission control — it's probabilistic guidance, same failure mode as §46/§47 above.

33. Piped Stdin Size Limit in Claude Code — 10 MB Cap

Claude Code enforces a hard 10 MB limit on piped stdin; exceeding it exits immediately with an error rather than truncating. For large inputs, reference the file path instead of piping — `claude -p 'explain the root cause in build-error.txt'` lets Claude `Read` the file directly, bypassing the stdin cap.

34. `claudeMdExcludes` — Personal Monorepo Filtering

Set in `.claude/settings.local.json` (gitignored, personal) to exclude sibling-package CLAUDE.md files from loading, without affecting teammates who still need them. Setting it in the committed `.claude/settings.json` instead would affect the whole team — the wrong file for a personal preference.

35. CLAUDE.md `@path` Import Syntax

`@README` or `@package.json` in CLAUDE.md/skill Markdown **imports that file's contents** into context at that point, not just a textual mention. To reference a path as visible prose *without* triggering an import, wrap it in a code span: ``@docs/legacy-notes.md``.

49. Plan Mode vs. Direct Execution

Task Statement 3.4.

Signal	Use plan mode	Use direct execution
Scope	Multi-file, architectural (e.g. monolith → microservices, a 45+ file library migration)	Single-file, well-understood (a clear stack trace, one validation check)
Number of valid approaches	Multiple, with real tradeoffs (different infra requirements)	One obvious approach
Cost of getting it wrong	High — rework across many files	Low — easy to fix
Reversibility	Committing to a structure is hard to unwind later	Trivially reversible

Exam sample-question framing: a monolith→microservices restructuring task should enter plan mode to explore dependencies and design service boundaries *before* any changes — starting with direct execution and letting boundaries emerge from incremental changes risks costly rework once dependencies surface late; comprehensive upfront instructions without exploration assume you already know the right structure; waiting to switch to plan mode only if "unexpected complexity" appears ignores that the complexity is already known from the task description itself, not something that might emerge later.

Combining both

Plan mode for investigation and design, then direct execution to implement the approved plan — not an either/or choice for a task that has both an exploratory phase and a mechanical implementation phase.

The `Explore` subagent

Use a dedicated `Explore` subagent for verbose discovery phases (mapping a large codebase, scanning many files for a pattern) so the exploration's noisy intermediate output doesn't fill the main conversation's context — only the summary returns. See §16 above for the `context: fork + agent: Explore` combination that achieves this inside a skill.

50. Iterative Refinement Techniques

Task Statement 3.5.

Concrete input/output examples beat prose descriptions

When a transformation is described only in prose ("normalize the date format"), the model can interpret it inconsistently across edge cases. Two to three concrete input→output examples pin down the exact expected transformation far more reliably than more prose:

```
Input:  "March 3rd, 2026"      → Output: "2026-03-03"
Input:  "3/3/26"              → Output: "2026-03-03"
Input:  "sometime in early 2026" → Output: null (ambiguous – do not guess)
```

Test-driven iteration

Write the test suite first (expected behavior, edge cases, performance requirements), implement, then iterate by sharing the specific **test failures** — not a vague "it's not working" — to guide the next revision. This gives the model a concrete, checkable target rather than open-ended re-interpretation.

The interview pattern

Before implementing in an unfamiliar domain, have Claude **ask clarifying questions first** (cache invalidation strategy, failure-mode expectations) rather than silently picking a default and building on an assumption the developer may not have anticipated needing to state.

Batching interacting issues vs. sequential fixes

Situation	Approach
Multiple issues that interact (fixing one changes the correct fix for another — e.g. two related null-handling bugs in the same function)	Provide all of them in one detailed message so the model reasons about them jointly
Multiple independent issues (unrelated files, no interaction)	Fix sequentially — one at a time, verify, move to the next

Specific test cases for edge-case gaps (Scenario 6: extraction)

When a null-value edge case is mishandled in a migration script, give the specific input and expected output for that case, not a restated general instruction — "handle nulls better" reproduces the same ambiguity that caused the miss in the first place.

51. Integrating Claude Code into CI/CD Pipelines

Task Statement 3.6.

`-p / --print` — non-interactive mode

```
# ❌ Hangs — waiting for interactive input in a pipeline with no human present
claude "Analyze this pull request for security issues"

# ✅ Correct — non-interactive, exits after producing output
claude -p "Analyze this pull request for security issues"
```

`CLAUDE_HEADLESS=true` and a `--batch` flag are **not real** — `-p / --print` is the only documented mechanism. Redirecting stdin from `/dev/null` is a workaround, not the intended fix, and doesn't address the actual cause (Claude Code waiting for interactive input, not a stdin-format issue).

Structured JSON output for automated PR comments

```
git diff origin/main | claude -p 'review this PR for security issues' \
  --output-format json --json-schema review-findings.schema.json
```

`--output-format json` emits a JSON object (including `session_id` and cost fields); `--json-schema` constrains the output to a given JSON Schema shape — together, this is the mechanism for producing machine-parseable structured findings a CI step can post as inline PR comments, rather than parsing free-text output.

CLAUDE.md as the CI context-injection mechanism

Document testing standards, fixture conventions, and review criteria in `CLAUDE.md` so CI-invoked Claude Code sessions have the same project context an interactive session would — e.g. "these are the fixtures already available; don't suggest tests that duplicate existing coverage in `tests/fixtures/`".

Session isolation for review — don't let the generator review itself

The same session that generated a code change is measurably less effective at reviewing it than an independent instance — see §14, "Multi-Instance and Multi-Pass Review Architectures", in [04-prompt-engineering-and-structured-output.md](#) for the full mechanism and why (anchoring bias).

Matching API/CLI approach to workflow latency (batch vs. blocking)

A **blocking** pre-merge check and a **non-blocking** overnight report have different correct backing mechanisms — see §30, "Message Batches API and Batch Processing Strategy", in [04-prompt-engineering-and-structured-output.md](#) (Message Batches API has no latency SLA and cannot be used for a check developers are actively waiting on).

Avoiding duplicate findings across re-runs

Include prior review findings in context on a re-run (after new commits) and instruct Claude to report only new or still-unaddressed issues — otherwise every re-run re-flags everything from scratch, drowning genuinely new findings in noise.

Domain 4 — Prompt Engineering & Structured Output

Exam weight: 20%. Covers Task Statements 4.1–4.6: explicit criteria to reduce false positives, few-shot prompting, `tool_use` + JSON schema enforcement, validation/retry/feedback loops, batch processing strategy, and multi-instance/multi-pass review architectures.

Running example used throughout: *Scenario 5 — Claude Code for CI/CD* (false-positive-sensitive review criteria) and *Scenario 6 — Structured Data Extraction* (schema design, validation loops, batch processing).

31. Explicit Criteria to Improve Precision and Reduce False Positives

Task Statement 4.1.

Specificity trades false positives against false negatives — there is no free lunch

Criteria style	False positives	False negatives
Broad / open-ended ("flag anything that looks like a security risk")	High — flags superficially risky but benign code	Low — wide net
Narrow / specific and checkable ("flag code that concatenates unsanitized user input into a SQL string")	Low — only fires on an objectively true condition	Higher — misses issues outside the named categories

Choose based on which failure mode is costlier for the use case: a lint-style gate blocking CI should minimize false positives (developer trust erodes fast); a first-pass triage a human reviews afterward can tolerate more false positives in exchange for fewer misses.

Vague instructions don't improve precision — even when they sound cautious

"Be conservative" and "only report high-confidence findings" are **not** specific categorical criteria — they ask the model to self-assess confidence, which is exactly as unreliable here as self-reported confidence is for escalation decisions (see §58 in [05-context-management-and-reliability.md](#)). The fix that actually moves precision is naming the **concrete, checkable condition**:

- ✗

"Check that comments are accurate."
- ✓

"Flag a comment only when the claimed behavior directly contradicts the function's actual code behavior (e.g. comment says 'returns null on error' but the function throws). Do NOT flag comments that are merely incomplete, outdated in wording, or stylistically loose."

Explicit severity criteria with concrete examples per level

```
<critical>
  Confirmed security vulnerability or data-loss bug.
  Example: `query = "SELECT * WHERE id=" + user_input` → critical.
</critical>
<high>
  Crash-inducing logic error.
```

```
Example: array index starts at 1 against a 0-indexed array → high.
</high>
<low>
Style-only: naming, missing docstring, line length.
</low>
```

Named XML tags (`<critical>`, `<high>`, `<low>` — see also `<security_criteria>`, `<correctness_criteria>` style tags) give each category an unambiguous boundary Claude respects, versus continuous prose or a single unheaded bullet list where criteria can bleed across categories.

Temporarily disabling a high-false-positive category to restore trust

If one review category (e.g. "possible race condition") has a high false-positive rate, developers stop trusting **all** categories from the tool, including accurate ones. The correct move is to disable that specific category while its prompt is improved — not to soften every category's criteria, which degrades the categories that were already working.

19. Prompt Structuring — XML Tags for Category Isolation

Task Statement 4.1 (supporting technique). When a prompt defines multiple distinct categories — each with its own criteria, rules, and examples — **uniquely named** XML tags create unambiguous boundaries that prevent cross-contamination between them.

```
<security_criteria>
  Flag: hardcoded secrets, SQL injection, missing auth checks.
  Example: `query = "SELECT * WHERE id=" + user_input` → critical.
</security_criteria>

<correctness_criteria>
  Flag: off-by-one errors, null dereferences, wrong return types.
  Example: loop index starts at 1 on a 0-indexed array → high.
</correctness_criteria>

<style_criteria>
  Flag: inconsistent naming, missing docstrings, lines > 120 chars.
  Severity: all style issues = low.
</style_criteria>
```

Claude is trained to respect XML tag boundaries as semantic containers — criteria inside `<security_criteria>` apply only to security, and won't bleed into how correctness or style issues are judged.

Approach	Problem
Continuous prose	No structural boundary — criteria can blend across categories
Single bullet list, no headers	Bullet order implies grouping but is ambiguous
Repeat full criteria in each section	Inflates prompt size; copies can drift and contradict
Generic, repeated tag names (<code><criteria>...</criteria></code> used twice)	Weaker — multiple instances of the same tag name lose distinguishing meaning

Use named XML tags whenever a prompt has multiple **independent** sections that must not bleed into each other: criteria categories, evaluation dimensions, persona constraints, step-by-step instructions.

54. Few-Shot Prompting for Output Consistency

Task Statement 4.2.

When few-shot beats more detailed prose

Detailed instructions alone often produce inconsistent formatting or inconsistent handling of ambiguous cases. Two to four **targeted** examples — especially ones that show *why* one action was chosen over a plausible alternative — generalize the model's judgment to novel cases better than an exhaustive rule list, because the model learns the underlying reasoning pattern, not just a lookup table of pre-specified cases.

```
Example 1 – customer states an order number in the same message as their name:  
Query: "Hi, I'm Dana Kim, checking on order #58291"  
Action: call get_customer first (verify identity), THEN lookup_order  
Why: an order number alone is not sufficient to authorize account actions  
      without a verified customer_id – even though the order number was given.
```

Demonstrating desired output format consistently

```
{location, issue, severity, suggested_fix} ← every finding uses exactly this shape
```

Showing 2–3 findings in this exact shape is more reliable than describing the shape in prose, especially for getting severity labeling consistent across many findings.

Reducing hallucination in extraction (Scenario 6)

Few-shot examples that show correct extraction from **varied document structures** (inline citations vs. a bibliography section, a narrative paragraph vs. an embedded table) teach the model to generalize the extraction pattern across formats it hasn't seen verbatim — this is what fixes empty/null extraction on documents whose layout differs from the most common training-adjacent shape, more reliably than adding more prose describing "handle different formats."

Distinguishing acceptable patterns from genuine issues (false-positive reduction)

Pairing a "this looks risky but is actually fine, don't flag it" example with a "this is genuinely the issue, flag it" example in the same few-shot set teaches the boundary directly — this is often more effective than a purely negative instruction list of things not to flag.

55. Enforcing Structured Output with `tool_use` and JSON Schemas

Task Statement 4.3. Builds on §1, "Tool Choice", in [00-foundations-messages-api.md](#).

`tool_choice` modes for structured-output guarantees

Mode	Guarantees a tool fires?	Use when
<code>auto</code>	✗ No	A plain-text fallback is acceptable
<code>any</code>	✓ Yes, but model picks which tool	Multiple extraction schemas exist and the document type is unknown ahead of time
<code>{"type": "tool", "name": "..."}"</code>	✓ Yes, this exact tool	A specific extraction (e.g. <code>extract_metadata</code>) must run before any enrichment step

```
# Document type unknown ahead of time – guarantee SOME structured extraction fires
response = client.messages.create(
    model="claude-sonnet-4-6",
    tools=[invoice_schema_tool, receipt_schema_tool, purchase_order_schema_tool],
    tool_choice={"type": "any"},
    messages=[{"role": "user", "content": document_text}],
)
```

`strict` — eliminates syntax errors, not semantic errors

`tool_use` + a JSON schema (with `strict` enabled) guarantees the response is structurally valid — every required field present, every value the declared type, every enum value one of the allowed options. It does **not** guarantee the *content* is correct: a line-items array can still fail to sum to the stated total, or a value can land in the wrong field, while remaining perfectly schema-valid.

```
result = run_extraction_tool(invoice_text)
calculated_total = sum(item["amount"] for item in result["line_items"])
if abs(calculated_total - result["invoice_total"]) > 0.01:
    flag_for_human_review(result, calculated_total) # semantic check – schema can't do this
```

Schema design: required/optional, nullable, and extensible enums

The fabrication problem: a field that is `required` *and* typed `number` forces Claude to invent a plausible value when the source document doesn't actually contain one — silently corrupting data.

```
# ✗ Forces fabrication when data is genuinely absent
"required": ["square_footage"], "properties": {"square_footage": {"type": "number"}}

# ✓ Correct – not required, plus a companion source/confidence enum
"required": ["address", "price"], # square_footage removed from required
"properties": {
    "square_footage": {"type": "number"},
    "square_footage_source": {"type": "string", "enum": ["stated", "estimated", "unknown"]}
}
```

Extend enum categories safely with an `"other"` + free-text detail pattern, and mark genuinely ambiguous cases explicitly rather than forcing a best-guess enum value:

```
"document_type": {"type": "string", "enum": ["invoice", "receipt", "purchase_order", "other"]},
"document_type_detail": {"type": "string", "description": "Only used when document_type is
'other'."},
"amount_clarity": {"type": "string", "enum": ["clear", "unclear"]}
```

Format normalization rules alongside a strict schema

A strict schema controls *shape*; it does not normalize inconsistent source formatting (dates as "3/3/26" vs "March 3, 2026" vs "2026-03-03"). Pair the schema with explicit normalization instructions in the prompt (target format, how to handle ambiguity) — the schema and the normalization rule solve different problems and both are needed.

56. Validation, Retry, and Feedback Loops for Extraction Quality

Task Statement 4.4.

Retry-with-error-feedback

On a validation failure, the follow-up request should include the **original document**, the **failed extraction**, and the **specific validation error** — giving the model a concrete target to correct, rather than re-sending the same prompt and hoping for a different result:

```
retry_prompt = (
    f"Your previous extraction failed validation: {validation_error}\n\n"
    f"Original document:\n{document_text}\n\n"
    f"Your previous (invalid) extraction:\n{json.dumps(failed_extraction)}\n\n"
    "Correct the specific field(s) that failed validation."
)
```

When retry will and won't help

Failure type	Will retry fix it?
Format/structural mismatch (wrong date format, field in wrong shape)	✅ Yes — the information exists, the model just needs to re-express it
Information genuinely absent from the provided document	❌ No — no amount of retrying recovers data that was never in the input

Retrying a "genuinely absent" case wastes calls; the correct response is a nullable field (§55) with a `_source:` "unknown" marker, not a retry loop.

Semantic validation vs. schema syntax errors

A schema (with `strict`) eliminates syntax errors (missing fields, wrong types) at the API layer. Semantic errors — line items that don't sum to the stated total, a value that landed in the wrong field, a date that's internally inconsistent — require **your own post-extraction validation code**, run after the tool call returns:

```
result = run_extraction_tool(invoice_text)
result["calculated_total"] = sum(i["amount"] for i in result["line_items"])
result["conflict_detected"] = abs(result["calculated_total"] - result["stated_total"]) > 0.01
```

Feedback loops that improve the system, not just one document

Add a detected_pattern field to structured findings/extractions — a short label for what code construct or document shape triggered this result. When developers dismiss a finding as a false positive, aggregating detected_pattern values across dismissals surfaces which pattern is over-triggering, turning individual dismissals into a systematic input for improving the prompt, schema, or few-shot set — rather than each dismissal being a one-off with no lasting signal.

```
finding = {
  "issue": "Possible race condition on shared counter",
  "detected_pattern": "unlocked_increment_in_async_handler",  # ← aggregatable across many findings
  "severity": "medium",
}
```

30. Message Batches API and Batch Processing Strategy

Task Statement 4.5.

What it is, and its hard limitation

Fire-and-forget: submit a batch, it processes asynchronously (up to 24h), no guaranteed latency SLA, ~50% cost savings vs. synchronous calls. The Batches API does not support a live round-trip mid-request — if Claude needs an external tool result to finish reasoning, that result must already be embedded in messages[] at submission time; it cannot be supplied after the model emits tool_use.

Feature	Allowed in a batch request?
tools= parameter, and tool_use blocks in the response	✔ Yes
Pre-populated multi-turn history including prior tool_use / tool_result	✔ Yes
Pausing mid-request to inject a freshly-fetched tool result	✘ No

Matching workload to API — blocking vs. non-blocking (Scenario 5: CI)

Workload	Correct API
Blocking pre-merge check — a developer is waiting on the result before merging	Synchronous messages.create() — batch's uncapped latency (up to 24h) is unacceptable here regardless of the cost savings
Overnight technical-debt report, weekly audit, nightly test generation	Message Batches API — latency-tolerant, and the 50% saving compounds over routine/scheduled runs

A proposal to move both workflows to batch processing to capture the discount should be pushed back on for exactly this reason — cost savings do not override a hard latency requirement.

Failure handling by `custom_id`

Each batch request/response pair is correlated by a `custom_id` you supply — results can return in any order, so key by `custom_id`, never by position. On partial batch failure, resubmit only the failed `custom_id`s (with any needed fix, e.g. chunking a document that exceeded context limits) rather than resubmitting the entire batch.

Prompt refinement before scaling up

Refine the extraction/review prompt against a small sample **before** submitting a large batch — a prompt issue discovered only after processing 10,000 documents costs far more (in both money and turnaround time) to fix than one caught on a 20-document sample.

14. Multi-Instance and Multi-Pass Review Architectures

Task Statement 4.6.

Why self-review misses issues — anchoring bias

A Claude instance reviewing its own output within the same conversation is anchored to its own prior reasoning: it tends to justify its earlier decisions rather than question them. A fresh instance, with no attachment to those decisions, reviews the artifact as a stranger would — this is *not* about extended-thinking budget or a stricter self-review prompt; both leave the anchoring effect largely intact.

```
# Instance 1 – Generator
gen_response = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=2000,
    system="You are a senior developer. Write clean, efficient code.",
    messages=[{"role": "user", "content": "Write a Python function to parse JWT tokens"}],
)
generated_code = next(b.text for b in gen_response.content if b.type == "text")

# Instance 2 – Fresh reviewer, no shared conversation history
review_response = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=1000,
    system="You are a security-focused code reviewer. Find bugs and vulnerabilities.",
    messages=[{"role": "user", "content": f"Review this code for bugs:\n\n{generated_code}"},
])
```

Pass the raw artifact, not a generator-authored summary

A generator-written summary of its own changes frames the decisions in a self-justifying way; the reviewer then evaluates that framing rather than forming an independent judgment from the raw diff. If token budget is a real constraint, **truncate** the raw diff (fewer lines, changed hunks only) — truncation preserves independence; summarization loses it.

Input to reviewer	Independence preserved?
Raw diff / raw output (or a truncated version of it)	✅ Yes

Input to reviewer	Independence preserved?
Generator-authored summary	✗ No — anchored to the generator's framing
Generator's extended-thinking trace	✗ No — worst case, adopts the generator's entire reasoning chain

Multi-pass review for large changesets

For a large PR (14+ files), one combined pass produces inconsistent depth and even contradictory findings (the same pattern flagged in one file, approved in another) — this is attention dilution, covered in depth at §38, "Task Decomposition Strategies for Complex Workflows", in [01-agentic-architecture-and-orchestration.md](#). The fix is the same principle applied to review specifically: **per-file local-analysis passes, plus one separate cross-file integration pass** for data-flow and consistency issues — not a bigger model/context window (that doesn't fix attention *quality*), and not a "3 runs, keep only issues found twice" consensus filter (which actively suppresses real bugs that only manifest intermittently).

Confidence-annotated verification passes

Have a verification pass have the model self-report a confidence level alongside each finding — this is useful for **routing** (send low-confidence findings to a human, auto-apply high-confidence ones), which is different from using the same confidence number to decide *whether to escalate a customer case* (§58) — confidence as a review-routing signal and confidence as an escalation-trigger are not interchangeable uses of the same number.

Domain 5 — Context Management & Reliability

Exam weight: 15%. Covers Task Statements 5.1–5.6: preserving critical information across long interactions, escalation/ambiguity resolution, error propagation across multi-agent systems, context management in large codebase exploration, human review workflows/confidence calibration, and information provenance/uncertainty handling in multi-source synthesis.

Running example used throughout: *Scenario 1 — Customer Support Resolution Agent* (escalation calibration) and *Scenario 3 — Multi-Agent Research System* (error propagation, provenance, coverage gaps) and *Scenario 6 — Structured Data Extraction* (human review routing).

17. Managing Conversation Context to Preserve Critical Information

Task Statement 5.1.

The context eviction problem

Context is finite. In long sessions, early content gets compressed or evicted as the window fills — Claude does not "remember" facts from earlier turns once they fall outside active context, and may hallucinate a plausible-sounding replacement instead of admitting it doesn't know:

Hour 1: Claude reads code → "OrderService extends TransactionalBase" ← in context
Hour 4: Early turns compressed → Claude says "extends a standard base controller" ← hallucinated

This is a context-**capacity** problem, not a model-quality problem — increasing `max_tokens` (output only), or switching to a larger model mid-session (more parameters ≠ restoring evicted history), fixes neither.

Progressive summarization risk — losing precise values

Summarizing a long conversation risks condensing numerical values, percentages, dates, and customer-stated expectations into vague prose ("the customer mentioned some pricing concern") that loses exactly the detail later turns need. The fix is a persistent **"case facts" block** — extracted transactional facts (amounts, dates, order numbers, statuses) carried in every prompt *outside* the summarized history, not vulnerable to the summarization pass:

```
<case_facts>
  customer_id: CUST-88213
  order_id: #58291, ordered 2026-02-11, amount $142.50
  prior_offer: 10% loyalty discount (declined)
  customer_request: price-match a competitor's current $128 listing
</case_facts>
```

Trimming verbose tool outputs before they accumulate

A raw order-lookup response with 40+ fields, when only 5 are relevant to a return/refund flow, consumes context disproportionately to its value if passed through unfiltered. Trim to only the relevant fields before the result enters context — do this consistently, not just for the first call, since accumulated verbose results compound over a multi-turn session.

The "lost in the middle" effect

Models attend most strongly to content at the **beginning and end** of a long input; content buried in the middle of a large aggregated document (e.g. the third of five sections in a synthesized research report) is systematically under-attended regardless of its actual importance — this is a positional bias, not a quality signal about that content.

Fix: a key-findings summary at the very top of the aggregated document, plus explicit section headers throughout, so critical findings are seen in the high-attention zone regardless of where they physically sit in the body:

```
## Key Findings Summary
- Market sizing: TAM $4.2B, growing 12% YoY
- **Regulatory risk: GDPR Article 22 requires human review for automated decisions** ← surfaced early
---
## 1. Market Sizing
[full detail...]
## 3. Regulatory Risk
[full detail...]
```

Reordering sections so the important finding lands last, or splitting into two shorter documents without headers, treats a symptom rather than the structural cause.

60. Context Management in Large Codebase Exploration

Task Statement 5.4.

Scratchpad files as external memory

Have Claude write concrete findings to a file as it discovers them — the file persists outside the context window and can be re-read at any point, so facts survive context compression, instead of the model drifting into "typical pattern" guesses once the specific class it found earlier has been evicted:

```
findings.md
## OrderService
- Extends: TransactionalBase (custom, NOT a standard base controller)
- Located: src/services/order/OrderService.java:12
```

Subagent delegation for verbose discovery

Spawn a subagent for a specific, bounded discovery question ("find all test files," "trace the refund flow's dependencies") while the main/coordinator agent stays at a high level of coordination — this isolates verbose exploration output from the main conversation, same principle as `context: fork` for skills (§16 in [03-claude-code-configuration-and-workflows.md](#)). Summarize key findings from one exploration phase and inject that summary as the seed context for the next phase's subagents, rather than letting each new subagent re-explore from scratch.

`/compact` for mid-session context relief

Use `/compact` to manually trigger context compaction mid-session once verbose discovery output has filled a meaningful fraction of the window — the right tool when exploration itself (not conversation length generally) is what's crowding context.

Crash recovery via structured state manifests

For crash recovery in a long-running multi-agent job: have each agent export its state to a known location (a manifest) as it completes work; on resume, the coordinator loads the manifest and injects only what's needed to continue, rather than re-running completed work from scratch.

58. Escalation Triggers and Ambiguity Resolution

Task Statement 5.2.

Appropriate escalation triggers

Trigger	Escalate?
Customer explicitly asks for a human	✅ Immediately — honor the request without first attempting autonomous resolution
A genuine policy gap — the policy is silent on this exact request (e.g. competitor price-matching, when policy only covers own-site price adjustments)	✅ Yes — this is a policy exception requiring human judgment, not "just a complex case"
Agent cannot make meaningful progress (contradictory account data, tool failures with no fallback)	✅ Yes
A routine, well-covered case (standard damage replacement with photo evidence, squarely inside policy)	❌ No — resolve autonomously

Honoring an explicit request vs. offering to help first

If a customer's *first* message already demands a human, escalate immediately — do not spend a turn attempting resolution first. If the customer is frustrated but the issue is squarely within the agent's capability, **acknowledge the frustration and offer resolution**; escalate only if they reiterate the preference for a human after that offer. These are different situations with different correct responses — collapsing them into one rule (either "always try first" or "always escalate on any frustration signal") gets one of the two wrong.

Why self-reported confidence and sentiment are unreliable escalation proxies

A model's own confidence score reflects its *internal uncertainty*, not the case's actual complexity — these diverge in both directions:

Situation	Self-reported confidence	Actual complexity
Clear evidence, unfamiliar phrasing	Low	Low — still resolve directly
Genuinely ambiguous/contradictory evidence	Low	High — may warrant escalation

The correct signal is **evidence quality** (is the account data clear and unambiguous, does policy clearly address this case), not the magnitude of a self-reported number. A fixed threshold rule ("escalate if confidence < 50%") systematically over-escalates easy-but-unfamiliar-phrasing cases and under-escalates hard-but-familiar-phrasing ones. The same logic applies to sentiment-based escalation: frustration doesn't correlate with case complexity, so an escalation trigger tied to negative-sentiment detection solves a different problem than "is this case actually hard."

Multiple customer matches — ask, don't guess

When a lookup returns multiple matching customer records, the correct response is to **request an additional identifier** from the customer (e.g. "can you confirm the billing zip code?") — not to heuristically pick the most-likely match. Selecting by heuristic risks acting on the wrong account entirely, which is a far worse failure than asking one extra clarifying question.

25. Agent Escalation Design — Self-Reported Confidence, in Detail

Task Statement 5.2 (supporting detail for §58).

Why numeric thresholds on self-reported confidence are wrong

Setting "escalate if confidence < 50%" treats the score as objective ground truth about the case, when it is really a proxy for the model's own uncertainty about its training-data familiarity with the phrasing. No category of question (e.g. billing proration) is automatically exempt from or guaranteed to trigger confidence-based escalation — evidence quality is what matters in every category.

Building escalation logic around structured evidence, not raw confidence

```
escalation_signal = {
  "source_reliability": "high",      # was the account data internally consistent?
  "contradictions_found": False,    # did multiple sources/records disagree?
  "policy_match": "exact",          # "exact" | "partial" | "silent"
}
should_escalate = (
  escalation_signal["contradictions_found"]
  or escalation_signal["policy_match"] == "silent"
  or escalation_signal["source_reliability"] == "low"
)
```

This routes on the actual drivers of case difficulty, not a single scalar the model reports about itself.

59. Error Propagation Strategies Across Multi-Agent Systems

Task Statement 5.3. Complements the coordinator-side handling in §24, "Multi-Agent Architecture — Hub-and-Spoke Pattern", in [01-agentic-architecture-and-orchestration.md](#).

Structured error context enables intelligent recovery

A subagent reporting a failure should return: failure type, what was attempted, any partial results obtained, and possible alternatives — not a generic status string like "search unavailable", which hides everything the coordinator would need to decide what to do next.

```
# ✅ Structured – the coordinator can actually decide something from this
{
  "status": "error",
  "failure_type": "timeout",
  "attempted_query": "AI adoption statistics in independent music production 2025-2026",
  "partial_results": [...],      # whatever was retrieved before the timeout
  "suggested_alternative": "retry with a narrower date range",
}

# ❌ Generic – coordinator has nothing to act on
{"status": "search unavailable"}
```

Access failures vs. valid empty results (again, at the multi-agent level)

A timed-out subagent and a subagent that successfully found zero matching results are different outcomes requiring different coordinator responses — conflating them (e.g. by silently returning an empty list for both) either hides a real failure or, worse, marks it as success. See also §42 in [02-tool-design-and-mcp-integration.md](#) for the same distinction at the individual-tool-call level.

Two anti-patterns: silent suppression, and total abort

Anti-pattern	Why it fails
Catch the failure inside the subagent, return an empty result marked "success"	Coordinator believes the query succeeded with nothing to find — a false negative that corrupts downstream synthesis
Propagate any single subagent failure straight to a top-level handler that terminates the entire workflow	Discards all other subagents' completed work over one recoverable or non-critical failure

Correct pattern: proceed on partial results, annotate the gap

```
results, errors = {}, {}
for name, outcome in subagent_outputs.items():
    (errors if outcome.is_error else results)[name] = outcome.content_or_error_context

report = synthesize(results)
for name, ctx in errors.items():
    report.add_coverage_gap(name, reason=ctx)    # honest, not fabricated
```

Re-running every subagent from scratch after one failure wastes the other completed work for no gain; fabricating content to paper over the gap silently corrupts the report in a way readers cannot detect.

What subagents report for internally-resolved transient failures

If a subagent retries a transient failure internally and succeeds (e.g. a DB reconnect on attempt 3), it reports **success only** — not partial results, not `is_error`, not the retry history. The task completed; surfacing resolved noise to the coordinator adds nothing.

61. Human Review Workflows and Confidence Calibration

Task Statement 5.5.

Aggregate accuracy can hide segment-level failure

A 97% overall extraction accuracy figure can mask a document type or field that performs far worse than average — e.g. handwritten forms at 60% accuracy, buried inside an aggregate that looks healthy. Validate accuracy **by document type and by field**, not only in aggregate, before reducing human review based on the aggregate number.

Two distinct mechanisms — don't conflate them

Mechanism	Purpose	What it answers
Stratified random sampling of <i>high-confidence</i> extractions	Ongoing measurement — is the error rate on "confident" outputs actually as low as assumed? Are there novel error patterns emerging?	"Is our confidence calibration still trustworthy?"
Confidence-based / field-level-ambiguity routing	Which specific extractions get a human reviewer's attention, given limited reviewer capacity	"Which of <i>these</i> extractions needs a human right now?"

Routing review attention should be driven by **confidence scores, document characteristics (e.g. known-hard document types), and field-level ambiguity** — not by random sampling. Random sampling is the right tool for *measuring* the system's error rate (including on the high-confidence bucket, to catch calibration drift); it is the wrong tool for *deciding which items a limited reviewer pool should look at right now*, since it ignores exactly the signal (confidence, ambiguity) that would make review time efficient.

```
def route_for_review(extraction):
    if extraction["low_confidence_fields"]:
        return "human_review"                # field-level ambiguity signal
    if extraction["document_type"] in KNOWN_HARD_TYPES:
        return "human_review"                # document-characteristic signal
    if extraction["conflict_detected"]:
        return "human_review"                # see §56
    return "auto_approve"

def measure_error_rate_sample():
    # separate process — stratified RANDOM sample of "auto_approve" outputs,
    # to check whether confidence calibration still holds and catch novel error patterns
    return stratified_random_sample(auto_approved_extractions, strata="document_type")
```

Calibrating thresholds with labeled validation data

Field-level confidence scores are only useful once calibrated against a labeled validation set (does "confidence: 0.9" on this field actually correspond to ~90% correctness on held-out labeled examples?) — an uncalibrated raw confidence number from the model is not directly interpretable as a probability of correctness.

62. Information Provenance and Uncertainty in Multi-Source Synthesis

Task Statement 5.6.

Claim-source mappings must survive summarization

Source attribution is commonly lost exactly at the summarization step — a synthesis agent that compresses findings without preserving which source backs which claim leaves the final report unable to say where anything came from. Require every subagent to output **structured claim-source mappings** (claim, evidence excerpt, source URL/document name, and any page/section reference), and require the synthesis step to **preserve and merge** these mappings rather than re-summarizing them away:

```
finding = {
  "claim": "...",
  "evidence_excerpt": "...",
  "source_url": "https://example.org/report-2026",
  "document_name": None,
  "published_date": "2026-02-14",
}
```

Conflicting statistics from credible sources — annotate, don't arbitrate

When two credible sources report different numbers for the same metric, the correct handling is to **include both values, explicitly annotated with their sources**, and let the coordinator (or the reader) decide how to reconcile them — not to silently pick one, and not to average them into a number neither source actually reported.

```
## Market size estimates (conflicting)
- Source A (Industry Report 2026): $4.2B TAM
- Source B (Analyst Brief, Feb 2026): $3.1B TAM
*(Difference may reflect differing scope – Source A includes adjacent services.)*
```

Temporal data — dates prevent false "contradictions"

Two findings that look contradictory can simply be from different points in time. Requiring every subagent to include a publication/collection date in its structured output lets the synthesis step correctly interpret an apparent conflict as a temporal trend instead of a genuine disagreement.

Structuring the report — well-established vs. contested, and content-appropriate rendering

Explicit report sections should distinguish well-established findings (multiple consistent sources) from contested ones (conflicting sources, annotated as above) — and different content types should render in their natural form (financial data as tables, narrative findings as prose, technical/structured findings as lists) rather than flattening everything into one uniform format for consistency's sake.

Coverage-gap annotation ties back to §59

When a subagent's failure means a topic area has thin or no coverage, the synthesis output should say so explicitly (a "coverage gaps" section) rather than silently presenting a partial picture as if it were complete — this is the same honesty principle as annotating an unrecoverable subagent error in §59, applied to the final report a user actually reads.

Out-of-Scope Exam Topics

A deliberate anti-scope list. These are real, current Claude/Anthropic topics — some of them (newer model families, prompt caching mechanics, streaming) are already partially documented elsewhere in this knowledge base for non-exam purposes — but the **Claude Certified Architect – Foundations** exam guide

explicitly excludes them. Don't spend study time here; don't expect exam questions on them; and don't let newer live-API detail (e.g. from the `claude-api` skill) bleed into exam-scoped answers, since the exam targets `claude-sonnet-4-6` specifically (see §6 in [00-foundations-messages-api.md](#)).

63. Out-of-Scope Topics Quick Reference

Verbatim from the exam guide's Appendix, Section 17:

Topic	Note
Fine-tuning Claude models or training custom models	Not part of the Agent SDK / API / Claude Code / MCP surface tested
Claude API authentication, billing, or account management	Operational/account concerns, not architecture
Detailed implementation of specific programming languages or frameworks	Beyond what's needed for tool/schema configuration
Deploying or hosting MCP servers (infrastructure, networking, container orchestration)	The exam tests MCP <i>tool/resource design</i> and <i>client-side configuration</i> , not server hosting
Claude's internal architecture, training process, or model weights	Not testable from a practitioner's vantage point
Constitutional AI, RLHF, or safety training methodologies	Research/training topics, not deployment architecture
Embedding models or vector database implementation details	Out of scope even though relevant to some RAG systems
Computer use (browser automation, desktop interaction)	A real built-in tool (current type strings: <code>computer_20250124</code> , <code>computer_20251124</code> , <code>computer_toolset_20260801</code>), but not tested here
Vision/image analysis capabilities	Not tested, despite <code>image</code> being a real content-block type
Streaming API implementation or server-sent events	Not tested — the exam's tool-loop examples are all non-streaming
Rate limiting, quotas, or API pricing calculations	Operational, not architectural
OAuth, API key rotation, or authentication protocol details	Even though MCP <code>oauth / headersHelper</code> config is in scope (§44 in 02-tool-design-and-mcp-integration.md), the underlying auth <i>protocol</i> mechanics are not
Specific cloud provider configurations (AWS, GCP, Azure)	Bedrock/Vertex/Foundry specifics are out of scope
Performance benchmarking or model comparison metrics	Not testable knowledge for this certification
Prompt caching implementation details (beyond knowing it exists)	You should know caching <i>exists</i> as a cost lever; breakpoint placement, invalidation mechanics, etc. are not tested
Token counting algorithms or tokenization specifics	Not tested

Why this matters for how you study

The exam's "In-Scope Topics" list (guide Appendix, Section 17) and the Technologies/Concepts list are both scoped tightly around: the Agent SDK's agentic loop / hooks / subagents, MCP tool and resource design, Claude Code's configuration surface (CLAUDE.md, rules, skills, commands, plan mode), `tool_use` + JSON schema structured output, the Message Batches API, and context/reliability patterns (escalation, error propagation, provenance). Live-API features that post-date or sit outside that scope — newer model families (Opus 5, Sonnet 5, Fable 5/5.1), fast mode, task budgets, compaction betas, context editing, Managed Agents, the Tool Runner helper, `inference_geo`, Workload Identity Federation — are real and current (see the `claude-api` skill for that surface) but are **not** exam content. If a practice question or this reference ever seems to reference one of them, treat that as scope drift to flag, not something to study for this specific certification.

Anthropic API – Quick Mental Map

Scope note: this file maps the **general** API/Claude Code surface (including things outside exam scope, like Opus 5 / Sonnet 5 / Fable 5). For the **exam-scoped** structure (Claude Certified Architect – Foundations, target model `claude-sonnet-4-6`), see the exam domain map below and [anthropic_api_reference.md](#).

Exam Domain Map (Claude Certified Architect – Foundations)

Domain 1 – Agentic Architecture & Orchestration (27%)

- Agentic loop → `stop_reason` is the **ONLY** reliable stop signal (not text content, not `maxTurns` alone – §40)
- Hub-and-spoke → coordinator owns routing/retry/synthesis; spokes isolated (§24)
- Task tool → exam term; built-in tool the model calls to spawn subagents. RENAMED to "Agent" in current tooling – exam still says "Task"/"allowedTools must include Task" (§36)
- AgentDefinition → {description, prompt, tools, model, ...} – the `ClaudeAgentOptions(agents={...})` programmatic subagent config (§36)
- fork_session → `resume=<id>`, `fork_session=True` → NEW session id, copies history at fork point, diverges independently (§37)
- Hooks (Agent SDK) → `PreToolUse` (can block + rewrite input) vs `PostToolUse` (observe/normalize only, too late to block) (§21)
- Task decomposition → fixed pipeline (predictable) vs adaptive (open-ended) (§38)

Domain 2 – Tool Design & MCP Integration (18%)

- Tool description → THE tool-selection mechanism; thin descriptions → misrouting between similar tools (§41)
- MCP `isError` → structured `errorCategory/isRetryable`, not a generic "failed" string (§42)
- Tool distribution → 4-5 tools per agent, scoped to its role; too many degrades selection reliability (§43)
- MCP scope → project (`.mcp.json`, team) vs user (`~/.claude.json`, personal) precedence: managed > local > project > user > plugin > connectors (§44)
- Grep→Read incremental → build understanding step by step, don't read the whole repo upfront (§45)

Domain 3 – Claude Code Configuration & Workflows (20%)

- Decision: which mechanism?

- "always true, no exceptions" → hook / permissions.deny (deterministic)
- "usually true, guidance is fine" → CLAUDE.md / rules / skills (probabilistic) (§46-47)
- PostToolUse quality gate → lint/format/test after every edit, independent of model remembering to ask (§48)
- Plan mode → architectural/multi-file/multiple-valid-approaches; direct execution → single-file, well-understood (§49)
- CI integration → -p/--print (non-interactive) + --output-format json + --json-schema (structured, parseable findings) (§51)

Domain 4 – Prompt Engineering & Structured Output (20%)

- Specificity tradeoff → narrow criteria ↓ false positives, ↑ false negatives (§31)
- Few-shot → 2-4 targeted examples > more prose, for ambiguous cases (§54)
- tool_use + schema → strict:true eliminates SYNTAX errors only, never semantic ones (sums, wrong field) – validate those yourself (§55)
- Nullable fields → remove from required[] + add a _source enum, don't force the model to fabricate (§15/55)
- detected_pattern → tag findings so dismissal patterns become a feedback loop that improves the prompt/schema (§56)
- Message Batches API → 50% cheaper, ≤24h, NO mid-request tool round-trip; never for a blocking pre-merge check (§30)
- Multi-instance review → fresh instance > self-review (anchoring bias); per-file passes + one cross-file pass for big diffs (§14)

Domain 5 – Context Management & Reliability (15%)

- Escalation → honor an explicit "get me a human" immediately; policy GAPS escalate too, not just "hard" cases; self-reported confidence ≠ actual complexity (§58/25)
- Error propagation → structured {failure_type, attempted, partial_results}; access-failure ≠ valid-empty-result; never silently suppress, never abort the whole workflow (§59)
- Human review routing → confidence + doc-type + field-ambiguity ROUTES review; RANDOM sampling MEASURES error rate – different jobs (§61)
- Provenance → claim-source mappings must survive synthesis; conflicting stats → annotate both, don't arbitrate (§62)

General API / Claude Code Structural Map

Models (default: claude-opus-5; never append date suffixes)

- claude-fable-5 → most capable, \$10/\$50 per MTok, 1M ctx
- claude-opus-5 → default, \$5/\$25 per MTok, 1M ctx
- claude-sonnet-5 → \$2/\$10 per MTok, 1M ctx
- claude-sonnet-4-6 → \$3/\$15 per MTok, 1M ctx
- claude-haiku-4-5 → \$1/\$5 per MTok, 200K ctx

Request

- messages[] → conversation history (user / assistant turns)
- tools[] → tool definitions (built-in or local)
- tool_choice → auto | any | tool | none
- system → system prompt
- thinking → {type: "adaptive"} current; budget_tokens deprecated/rejected on 4.7+
- output_config → {effort: low|medium|high|xhigh|max, format: {...}}
- model, max_tokens → always required

Response

- content[] → text | tool_use blocks
- stop_reason → end_turn | tool_use | max_tokens | stop_sequence | pause_turn | refusal
- refusal only → stop_details.category populated (e.g. "cyber", "bio")
- usage → input_tokens, output_tokens

You send back (when `stop_reason == "tool_use"`)

- └─ `tool_result` → type, `tool_use_id`, content

Agentic loop (your code)

- └─ `maxTurns` → counter you manage around `messages.create()`
- └─ `turn` → one tool call + response cycle

CLI session

- └─ `--name` → set name at startup
- └─ `/rename` → rename during session
- └─ `--resume <name>` → resume by name (omit name for picker)

Skills (filesystem-based)

- └─ `~/.claude/skills/` → user-level, all projects
- └─ `.claude/skills/` → project-level, repo-scoped
- └─ `conflict` → project-level wins

Multi-instance review

- └─ `generator` → fresh `messages.create()` call
- └─ `reviewer` → separate `messages.create()`, no generation history
- └─ in Claude Code → new chat, subagent prompt, or `/code-review` skill

Session resumption (API)

- └─ NO `session_id` parameter – does not exist in the Anthropic API
- └─ API is stateless – model sees only what is in `messages[]` per request
- └─ `resume` = YOU load prior history from storage + inject into `messages[]`
- └─ Claude Code CLI `--resume` is different: Claude Code manages disk storage itself

Context window – long sessions

- └─ early turns get compressed/evicted as context fills
- └─ `max_tokens` = output limit only (does NOT expand input context)
- └─ larger model = more params, NOT restored history
- └─ fix: scratchpad file → write findings to disk → read back on demand

Path-scoped rules + symlinks

- └─ rules match on symlinked path AND canonical path
- └─ no extra absolute-path entries needed for symlinked checkouts
- └─ write rules as project-root-relative globs (`src/handlers/**/*.go`)

Prompt structuring – XML tags

- └─ named tags → unambiguous category scope, no cross-contamination
- └─ generic tags → weaker (multiple `<criteria>` tags lose meaning)
- └─ prose / bullets → no structural boundary, categories can bleed

Structured output – missing data

- └─ required field + wrong type → model fabricates values
- └─ fix: remove from `required[]` → model omits cleanly
- └─ companion enum (`_source`) → downstream filterability preserved

Claude Code permission modes

- └─ `default` → prompts user for unapproved actions (needs human)
- └─ `acceptEdits` → auto-approves file edits; prompts for everything else
- └─ `dontAsk` → denies silently without prompting (CI-safe)
- └─ `bypassPermissions` → allows everything (dangerous)
- └─ `dontAsk` + `permissions.allow` = deterministic CI (no hanging, no surprises)

MCP tool naming

- └─ `format` → `mcp__<server-name>__<tool-name>` (double underscore)
- └─ `max length` → 64 characters
- └─ `matcher regex` → `mcp__billing__(issue_refund|void_authorization|apply_credit)`

`.claude/` directory (project-level)

- └─ `CLAUDE.md` → always loaded, top priority
- └─ `rules/*.md` → no ``paths:`` field = unconditional load, same priority as `CLAUDE.md`
- └─ → has ``paths:`` field = loads only when a matching file is read/edited

- └─ skills/<n>/SKILL.md → slash-command-invoked prompt packages
- └─ agents/<n>.md → subagent definitions
- └─ agent-memory/<n>/ → persistent subagent memory
- └─ output-styles/ → custom system-prompt sections
- └─ settings.json → permissions, hooks, env vars (committed)
- └─ settings.local.json → personal overrides (gitignored)
- └─ .mcp.json → team-shared MCP server config
- └─ no dedicated hooks/ or commands/ folder – hooks live in settings.json, commands = skills/

Claude Code hooks

- └─ PreToolUse → runs BEFORE tool; can block OR rewrite input (updatedInput)
 - └─ shape → {hookSpecificOutput: {hookEventName, permissionDecision, ...}}
 - └─ allow → permissionDecision: "allow"
 - └─ allow+mutate → permissionDecision: "allow", updatedInput: {...}
 - └─ deny → permissionDecision: "deny", permissionReason: "..."
 - └─ ask → permissionDecision: "ask" (no "defer" value exists)
- └─ PostToolUse → runs AFTER tool, observational only (cannot block)
- └─ Notification → status events only, cannot block
- └─ Stop → end of turn, observational only
- └─ matcher → regex; scope hook to specific tool(s); no matcher = fires for all
- └─ updatedInput → does NOT propagate between hooks; each hook sees original input

.mcp.json env var expansion

- └─ \${VAR} → unset + no default → warning logged, literal "\${VAR}" text used (not blanked)
- └─ \${VAR:-default} → falls back to `default` if unset, everywhere
- └─ scope → works in command, args, env, url, AND headers (not just env)

MCP server authentication

- └─ headers → static, hand-rotated credential
- └─ headersHelper → runs script fresh per connection (dynamic tokens, Kerberos)
 - (also called apiKeyHelper)
- └─ oauth block → only when a real OAuth authorization server exists
- └─ transport (http/sse/stdio) → orthogonal to auth, never carries auth logic itself

MCP server scope precedence (duplicate server name across scopes)

- └─ managed (org) > local > project > user > plugin-provided > claude.ai connectors
- └─ winner takes the ENTIRE entry – no field-level merging
- └─ version control status (checked-in vs not) has NO effect on precedence

Kept in a separate file so the main reference sections can grow without renumbering.

Claude Code – Slash Commands Reference

Commands are recognized only at the **start** of a message.

Built-in = always available. **Skill** = AI-powered prompt loaded from `.claude/skills/` or bundled.

Most important commands are listed first within each section.

⚡ Daily Drivers — Reach For These First

`/clear`

Built-in | Wipe conversation history and start fresh.

When: Context is polluted, Claude is confused, or you're switching tasks entirely.

Why: Removes accumulated noise without closing the session. File state and project are untouched.

How: Type `/clear` — no arguments needed.

`/compact [instructions]`

Built-in | Compress conversation history into a summary to free up context.

When: Session is long and approaching context limits but you need continuity.

Why: Keeps Claude aware of what happened without burning the full token budget on verbatim history.

How:

```
/compact
/compact focus on the auth changes and ignore the test output
```

`/code-review [effort] [--fix] [--comment]`

Skill | Independent code review pass on the current diff.

When: Before committing — want a fresh-perspective quality check.

Why: Runs as a separate pass, not anchored to your generation reasoning. Catches bugs, simplifications, and efficiency issues.

How:

```
/code-review          # default effort
/code-review high      # broader coverage
/code-review --fix     # apply findings automatically
/code-review --comment # post as inline PR comments
```

`/model [model-name]`

Built-in | Switch the AI model mid-session.

When: Need a faster/cheaper model for simple tasks, or the most capable model for complex ones.

Why: Avoids starting a new session just to change model.

How:

```
/model                # interactive picker
/model claude-sonnet-4-6
/model claude-opus-5
```

`/usage`

Built-in | Show token and cost usage for the current session.

When: Want to understand how much context a task consumed.

Why: Helps optimize prompts and decide when to `/compact` or `/clear`.

How: Type `/usage` — no arguments needed.

`/rename "name"`

Built-in | Rename the current session.

When: Starting a significant task you'll want to resume later.

Why: Gives the session a human-readable handle for `--resume`.

How:

```
/rename "q3-security-audit"  
/rename "payment-refactor"
```

Context & Session Management

`/rewind`

Built-in | Step back to a previous point in the conversation.

When: Claude went down a wrong path and you want to undo recent turns.

Why: More precise than `/clear` — preserves earlier good context.

How: `/rewind` — interactive picker shows recent turns to revert to.

`/context`

Built-in | Show what's currently loaded in the context window.

When: Debugging why Claude seems to be missing something or acting on stale info.

Why: Makes the context window contents visible so you can identify what to remove or compact.

How: `/context`

`/btw [note]`

Built-in | Add a side-note to the conversation without making it a full user turn.

When: You want to inject a reminder or constraint mid-task without interrupting Claude's flow.

Why: Keeps the note out of the main conversation rhythm.

How:

```
/btw don't touch the legacy auth module  
/btw we're using Python 3.11 not 3.12
```

`/memory`

Built-in | Edit CLAUDE.md memory files that persist across sessions.

When: Want to add, update, or remove persistent instructions Claude uses in every session.

Why: Memory files load automatically — editing here affects all future sessions in this project.

How: `/memory` — opens memory editor.

`/resume [session-name]`

Built-in | Resume a previously named session.

When: Returning to a multi-day task.

How:

```
/resume "q3-security-audit"
/resume                    # interactive picker if name forgotten
```

`/fork`

Built-in | Fork the current conversation into a new branch.

When: You want to explore an alternative approach without losing the current thread.

Why: Lets you try two directions in parallel.

How: `/fork`

Project & Workspace

`/init`

Skill | Analyze the repo and generate a `CLAUDE.md` file.

When: Starting work in a new repository with no `CLAUDE.md`.

Why: Gives future Claude instances instant context about commands, architecture, and conventions.

How: `/init`

`/add-dir [path]`

Built-in | Add an additional directory to Claude's workspace.

When: Working across multiple repos or folders in the same session.

How:

```
/add-dir ../shared-lib
/add-dir /workspace/config
```

`/cd [path]`

Built-in | Change Claude's working directory.

When: Need to switch context to a different folder mid-session.

How:

```
/cd ../backend  
/cd /workspace/project-mm
```

`/plan`

Built-in | Enter planning mode — Claude drafts a plan before acting.

When: Complex multi-step task where you want to review and approve the approach before any changes are made.

Why: Prevents Claude from jumping straight into edits on tasks that need upfront alignment.

How: `/plan` then describe the task — Claude proposes steps, you approve.

`/permissions`

Built-in | View and manage tool permissions for the current session.

When: Need to check what Claude is and isn't allowed to do, or grant/revoke access.

How: `/permissions`

`/mcp`

Built-in | Manage MCP (Model Context Protocol) server connections.

When: Adding, removing, or checking status of MCP tool servers.

How: `/mcp` — interactive server management.

Code Quality Skills

`/simplify`

Skill | Review changed code for reuse, simplification, and efficiency — then apply fixes.

When: After completing a feature — pre-commit cleanup pass.

Why: Quality-only pass (not bug hunting). Finds premature abstractions, redundant code, verbose patterns.

How: `/simplify`

`/security-review`

Skill | Security-focused review of pending changes.

When: Before merging anything touching auth, input handling, data storage, or external APIs.

How: `/security-review`

`/diff`

Built-in | Show the current diff of changes.

When: Want to see exactly what's changed before reviewing or committing.

How: `/diff`

Configuration & Setup

`/config`

Built-in | Open Claude Code settings interactively (model, theme, editor mode, keybindings).

When: Changing preferences without editing JSON directly.

How: `/config` — menu-driven.

To enable vim keybindings: `/config` → Editor mode → Vim (vim is **not** a slash command itself)

`/update-config`

Skill | Configure automated behaviors, permissions, and env vars in `settings.json`.

When: "Whenever X happens, do Y" — these require hooks that only `settings.json` can provide.

Why: Claude cannot fulfill automated behaviors through memory alone; hooks in `settings.json` run at the harness level.

How:

```
/update-config      # then describe: "allow npm commands"
                    # "when Claude stops, show a desktop notification"
                    # "set DEBUG=true in env"
```

`/keybindings-help`

Skill | Customize keyboard shortcuts and chord bindings.

How:

```
/keybindings-help  # then: "rebind ctrl+s", "change the submit key"
```

`/fewer-permission-prompts`

Skill | Scan transcripts for common approved read-only tool calls and add them to the allowlist.

When: Claude keeps prompting for permission on the same safe operations every session.

How: `/fewer-permission-prompts`

Research & Reference

`/claude-api`

Skill | Load the Claude API reference into context before writing Anthropic SDK code.

When: About to write code that calls Claude — model IDs, pricing, params, streaming, tool use, caching.

Why: API details change. Load this before coding to avoid stale model IDs or deprecated patterns.

How: `/claude-api` then ask your question.

`/deep-research [topic]`

Skill | Multi-step research using web search and synthesis.

When: Need thorough answers on a topic rather than a quick lookup.

How:

```
/deep-research Claude prompt caching best practices
```

Diagnostics & Feedback

`/doctor`

Skill | Run a setup health check on Claude Code.

When: Tools not working, MCP servers not connecting, unexpected behavior.

How: `/doctor` — outputs a diagnostic report.

`/debug`

Skill | Debug mode — detailed logging and diagnostics for the current session.

When: Something is failing silently and you need visibility into what's happening.

How: `/debug`

`/status`

Built-in | Show current session status — model, context usage, active connections.

How: `/status`

`/help`

Built-in | List all available commands and skills.

When: Don't know what's available.

How: `/help`

`/bug`

Built-in | Report a Claude Code bug — opens GitHub issue pre-filled with session context.

How: `/bug`

Workflow & Automation

`/loop [interval] [command]`

Skill | Run a command repeatedly on an interval or self-paced.

When: Polling CI status, watching for changes, or running iterative tasks automatically.

How:

```
/loop 5m /code-review      # run every 5 minutes
/loop /babysit-prs        # self-paced, model sets interval
```

`/batch`

Skill | Run a task across multiple files or targets in parallel.

When: Applying the same change or check to many files at once.

How: `/batch` then describe the task and targets.

`/background`

Built-in | Run a task in the background while continuing the current session.

When: Long-running tasks that don't need your immediate attention.

How: `/background` followed by the task description.

`/tasks`

Built-in | View and manage background tasks.

How: `/tasks`

`/goal [description]`

Built-in | Set an explicit goal for the session that Claude tracks throughout.

When: Long sessions where you want Claude to stay oriented to the primary objective.

How:

```
/goal migrate all API calls from v1 to v2 without breaking tests
```

`/effort [level]`

Built-in | Set reasoning effort level for the session.

When: Simple tasks don't need maximum reasoning; complex tasks benefit from it.

How:

```
/effort low      # faster, lighter reasoning
/effort high     # more thorough reasoning
```

Account & Integrations

`/login` / `/logout`

Built-in | Authenticate or sign out of Claude Code.

`/upgrade`

Built-in | Upgrade your Claude Code plan.

`/install-github-app`

Built-in | Install the Claude GitHub App for PR review integration.

`/install-slack-app`

Built-in | Install Claude Tag in a Slack workspace.

`/import`

Built-in | Import configuration from another agent tool (Codex, Gemini, etc.).

Quick Reference Table

Command	Type	Best for
<code>/clear</code>	Built-in	Reset context — confused Claude or task switch
<code>/compact</code>	Built-in	Compress long session to free context
<code>/code-review</code>	Skill	Pre-commit independent quality pass
<code>/model</code>	Built-in	Switch model mid-session
<code>/usage</code>	Built-in	Check token/cost usage
<code>/rename</code>	Built-in	Name a session for later resume
<code>/rewind</code>	Built-in	Undo recent turns

Command	Type	Best for
<code>/context</code>	Built-in	Inspect what's in context window
<code>/btw</code>	Built-in	Inject side-notes without a full turn
<code>/memory</code>	Built-in	Edit persistent CLAUDE.md memory
<code>/resume</code>	Built-in	Return to a named session
<code>/fork</code>	Built-in	Branch conversation to try alternate approach
<code>/init</code>	Skill	Generate CLAUDE.md for new repo
<code>/add-dir</code>	Built-in	Add another directory to workspace
<code>/cd</code>	Built-in	Change working directory
<code>/plan</code>	Built-in	Review+approve plan before Claude acts
<code>/permissions</code>	Built-in	View/manage tool permissions
<code>/mcp</code>	Built-in	Manage MCP server connections
<code>/simplify</code>	Skill	Post-feature cleanup pass
<code>/security-review</code>	Skill	Pre-merge security check
<code>/diff</code>	Built-in	Show current changes
<code>/config</code>	Built-in	Change settings (model, theme, editor mode)
<code>/update-config</code>	Skill	Automate behaviors via hooks, set env vars
<code>/keybindings-help</code>	Skill	Customize keyboard shortcuts
<code>/fewer-permission-prompts</code>	Skill	Reduce repetitive approval dialogs
<code>/claude-api</code>	Skill	Load API reference before writing SDK code
<code>/deep-research</code>	Skill	Multi-step web research
<code>/doctor</code>	Skill	Health check when Claude Code misbehaves
<code>/debug</code>	Skill	Detailed diagnostics for silent failures
<code>/status</code>	Built-in	Session status — model, context, connections
<code>/help</code>	Built-in	List all commands
<code>/bug</code>	Built-in	Report a Claude Code bug
<code>/loop</code>	Skill	Recurring/self-paced repeated tasks
<code>/batch</code>	Skill	Same task across many files in parallel
<code>/background</code>	Built-in	Run long task without blocking session
<code>/tasks</code>	Built-in	View background tasks
<code>/goal</code>	Built-in	Set session-level objective Claude tracks
<code>/effort</code>	Built-in	Set reasoning effort level

Source: Anthropic Claude Code official documentation. Commands verified August 2026.